Fraunhofer IPA

HOCHSCHULE HEILBRONN

Bachelor Thesis

Hochschule Heilbronn Fakultät für Informatik

Bachelor of Science (B. Sc.) Software Engineering

Entwicklung und Benchmarking einer Abstraktionsschicht für Quanten-Software

vorgelegt von

Daniel Faigle

Matrikelnummer: 215467

Partnerunternehmen:	Fraunhofer-Institut IPA
Erstprüfer:	Prof. Dr. rer. nat. David Kreplin (Hochschule Heilbronn)
Zweitprüfer:	M.Sc. Moritz Willmann (Fraunhofer IPA)
Zeitraum:	1.04.2026 - 03.08.2026

Abstract

This bachelor's thesis addresses the development and runtime analysis of an abstraction layer that allows quantum circuits and observables to be described in a single form and subsequently executed as objects of the quantum frameworks *PennyLane* and *Qiskit*. The software component was integrated into the *Executor* Python library, which is currently under active development at Fraunhofer IPA and is to be published under an open-source license in the near future. Its previous internal representation relied on *Qiskit* objects and was therefore tied to a single framework. This dependency is replaced by a self-contained data structure, so that the description of the circuit remains unchanged and the execution environments of the respective other framework can be addressed solely by selecting the backend. Since framework calls occur exclusively within the backends, a program written once also remains executable across framework version changes. In addition, the runtime overhead introduced by this layer compared to direct use of the frameworks is evaluated.

Zusammenfassung

Diese Bachelorarbeit befasst sich mit der Entwicklung und der Laufzeitanalyse einer Abstraktionsschicht, über die sich Quantenschaltkreise und Observablen einheitlich beschreiben und anschließend in den Objekten der Quantenframeworks *PennyLane* und *Qiskit* ausführen lassen. Die Softwarekomponente wurde in die Python-Softwarebibliothek *Executor* integriert, die derzeit vom Fraunhofer IPA aktiv weiterentwickelt wird und in naher Zukunft unter einer Open-Source-Lizenz veröffentlicht werden soll. Bislang stützte sich deren interne Darstellung auf *Qiskit*-Objekte und war damit an ein einzelnes Framework gebunden. Diese Bindung wird durch eine eigenständige Datenstruktur ersetzt, sodass die Beschreibung des Schaltkreises unverändert bleibt und sich allein durch die Auswahl des Backends die Ausführungsumgebungen des jeweils anderen Frameworks ansprechen lassen. Da Framework-Aufrufe ausschließlich innerhalb der Backends auftreten, bleibt ein einmal geschriebenes Programm zudem über Versionswechsel der Frameworks hinweg lauffähig. Ergänzend wird der zeitliche Mehraufwand bewertet, den diese Schicht gegenüber der direkten Nutzung der Frameworks verursacht.

Inhaltsverzeichnis

Abstract I

Zusammenfassung II

1. Einleitung 2

2. Theoretischer Hintergrund 4
2.1. Superposition 4
2.2. Notation von Quantenzuständen 5
2.3. Verschränkung 5
2.4. Bloch-Sphäre 6
2.5. Quantenschaltkreise und Gatter 7
2.6. Observablenberechnung 13

3. Quantencomputing-Software 14
3.1. PennyLane 14
3.2. Qiskit 15
3.3. Executor 15
3.4. Stand der Technik 17

4. Umsetzung und Implementierung der Abstraktionsschicht 18
4.1. Ausgangslage 18
4.2. Wahl der internen Objekte 18
4.3. Implementierung der Abstraktion 22
4.4. Integration in den Executor 25

5. Vorteile der Abstraktionsschicht 27
5.1. Einheitliche Definitionsschnittstelle 27
5.2. Modularisierbarkeit 28
5.3. Optionale Framework-Abhängigkeiten 29
5.4. Entkopplung von Framework-Versionen 30
5.5. Einschränkungen 31

6. Evaluation und Ergebnisse **32**
6.1. Aufbau der Messung 32
6.2. PennyLane und Executor im Vergleich 33
6.3. Qiskit und Executor im Vergleich 35
6.4. Ergebnisse des Vergleichs von PennyLane und Executor 38
6.5. Ergebnisse des Vergleichs von Qiskit und Executor 42

7. Fazit **45**
7.1. Zusammenfassung 45
7.2. Ausblick 47

Literaturverzeichnis **48**

A. Anhang **51**
A.1. Zugang zum Quellcode 51
A.2. Eidesstattliche Erklärung 52

Eidesstattliche Erklärung **52**

1. Einleitung

Mit dem Gedankenexperiment einer Katze, die zugleich lebendig und tot ist, veranschaulichte Erwin Schrödinger 1935 [17] eine der verblüffendsten Konsequenzen der Quantenmechanik. Phänomene der Quantenphysik wie die Superposition, bei der sich ein Teilchen gleichzeitig in mehreren Zuständen befinden kann, oder die Verschränkung, durch die Teilchen über beliebige Distanzen hinweg miteinander korreliert bleiben, widersprechen der Alltagserfahrung grundlegend. Genau von diesen Effekten erhofft man sich im Bereich des Computings einen fundamentalen Fortschritt [20]. Das entsprechende Feld nennt sich Quantencomputing und nutzt die Superposition, um Systeme zu bauen, deren kleinste Recheneinheiten nicht nur die klassischen Zustände 0 und 1 annehmen können, sondern beide zugleich.

Die Forschung an Quantencomputern hat in den vergangenen Jahren erheblich an Aufschwung gewonnen. Sowohl große Technologiekonzerne wie IBM, Google und Microsoft als auch spezialisierte Unternehmen wie IonQ oder D-Wave und zahlreiche Universitäten investieren beträchtliche Mittel in die Entwicklung leistungsfähiger Quantensysteme und Quantensoftware [5], [8]. Auch staatliche Stellen stufen die Entwicklung von Quantencomputern als strategisch relevant ein. Der Bund stellte seit 2017 rund eine Milliarde Euro allein für das Quantencomputing bereit. Über sämtliche Quantentechnologien hinweg summieren sich die öffentlichen Zuwendungen von Bund, Ländern und EU auf über 1,9 Milliarden Euro **bundestag2025quantentechnologien**. Dies zeigt, dass es sich um mehr als ein theoretisches Konzept handelt. Vielversprechende Anwendungsfelder liegen unter anderem in der Simulation von Molekülen für die Arzneimittelentwicklung [16], in Optimierungsproblemen aus Finanzwesen und Logistik, in der Kryptographie [5] sowie im maschinellen Lernen [4]. Dabei handelt es sich um Problemklassen, bei denen klassische Rechner an praktische Grenzen stoßen.

Zum Programmieren dieser Quantencomputer werden größtenteils Python-basierte Frameworks genutzt [7], mit denen sich sogenannte Schaltkreise und Gatter definieren lassen, die die Quantenbits manipulieren. In der täglichen Arbeit am Fraunhofer-Institut für Produktionstechnik und Automatisierung wird nicht nur eines dieser Frameworks eingesetzt, da sie jeweils unterschiedliche Vor- und Nachteile aufweisen. Diese Arbeit beschränkt sich auf die Frameworks *PennyLane* und *Qiskit*, die zu

den populärsten zählen [21] und auch bei Fraunhofer häufig verwendet werden. Beide nutzen zur Ausführung unterschiedliche Backends und verwenden verschiedene Notationen. Daraus ergibt sich das Problem, dass sich dasselbe Programm nicht auf beliebigen Backends ausführen lässt, sondern zunächst auf die jeweilige Notation umgeschrieben werden muss.

Am Fraunhofer IPA wurde hierfür ein Executor entwickelt, eine Software, die mehrere Frameworks bündelt und Schaltkreise an das jeweils gewünschte Backend weiterleitet. In der bestehenden Version werden Gatter und Qubits über einen Wrapper definiert, der *Qiskit*-Objekte in die Objekte der einzelnen Frameworks übersetzt. Dadurch ist der Executor eng an *Qiskit* gebunden, was der eigentlichen Aufgabe des Executors, nämlich der Framework-Unabhängigkeit, entgegensteht.

Ziel dieser Arbeit ist es, diese Bindung durch eine framework-unabhängige Abstraktionsschicht innerhalb des Executors aufzulösen. Über diese lassen sich Schaltkreise und Gatter einheitlich definieren, die anschließend automatisch an das gewünschte Backend weitergeleitet werden. Den Nutzerinnen und Nutzern bleibt dadurch das wiederholte Neudefinieren beim Wechsel des Frameworks erspart, und sie können innerhalb desselben Programms verschiedene Frameworks verwenden. Über diese Vereinheitlichung hinaus entstehen weitere Vorteile. Da ein Schaltkreis nur einmal beschrieben und unverändert auf unterschiedlichen Backends ausgeführt wird, beruhen Backend-Vergleiche auf einer kontrollierten Problemformulierung. Zudem lassen sich die einzelnen Frameworks unabhängig voneinander installieren, und Änderungen an deren Schnittstellen bleiben auf eine zentrale Schicht begrenzt, sodass ein einmal geschriebenes Programm über Versionswechsel hinweg lauffähig bleibt.

Das Vorgehen und die Struktur der weiteren Kapitel orientieren sich an drei Forschungsfragen. Zunächst wird untersucht, wie sich eine solche Abstraktionsschicht entwerfen und implementieren lässt (FF1). Darauf aufbauend wird bewertet, welche Vorteile die Abstraktionsschicht gegenüber der direkten Nutzung der einzelnen Frameworks bietet (FF2). Abschließend wird analysiert, welcher Zeit-Overhead durch den Executor im Vergleich zur direkten Nutzung von *PennyLane* und *Qiskit* entsteht (FF3).

2. Theoretischer Hintergrund

Um das Verständnis dieser Arbeit zu erleichtern, werden in diesem Kapitel zunächst die für das Quantencomputing relevanten quantenmechanischen Effekte und Notationen sowie die Grundlagen von Quantenschaltkreisen und Quantengattern vermittelt.

2.1. Superposition

Im Gegensatz zum klassischen Bit, das entweder den Wert 0 oder 1 annimmt, bietet das Quantencomputing den Vorteil der Superposition, die durch Quantenbits, auch Qubits genannt, realisiert wird [14]. Der Zustand eines Qubits ist eine Linearkombination der beiden Basiszustände $|0\rangle$ und $|1\rangle$ und kann diese vor der Messung gleichzeitig repräsentieren. So spannen n Qubits einen Zustandsraum von 2^n Basiszuständen auf. Da eine Messung jedoch nur einen einzelnen Basiszustand liefert, sind diese Zustände nicht direkt auslesbar. Dieses Prinzip bildet eine der fundamentalen Grundlagen des Quantenvorteils gegenüber der klassischen Datenverarbeitung [15]. Dadurch lassen sich Zustände abbilden, die nicht mehr auf $|0\rangle$ und $|1\rangle$ beschränkt sind, wodurch sich Algorithmen entwerfen lassen, die bestimmte Probleme schneller lösen als bekannte klassische Verfahren.

2.2. Notation von Quantenzuständen

Den Zustand eines Qubits beschreibt man sowohl im Quantencomputing als auch in der Quantenmechanik in der sogenannten Dirac-Notation, auch Bra-Ket-Notation genannt. Ein Zustandsvektor wird dabei als Ket $|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$ geschrieben. Hierbei sind α und β komplexe Zahlen und werden Amplituden genannt. Es gilt stets die Normierungsbedingung $|\alpha|^2 + |\beta|^2 = 1$, das heißt, die Betragsquadrate ergeben summiert die Wahrscheinlichkeit 1. Wird der Zustand gemessen, so ergibt sich der Wert 0 mit der Wahrscheinlichkeit $|\alpha|^2$ und der Wert 1 mit der Wahrscheinlichkeit $|\beta|^2$.

2.3. Verschränkung

Ein weiteres fundamentales Phänomen, durch welches sich im Quantencomputing ein Vorteil verschaffen lässt, ist die Verschränkung von Qubits. Bei einem verschränkten reinen Zustand mehrerer Qubits ist der Gesamtzustand mehr als die Summe der einzelnen Zustände. Er lässt sich nicht als Tensorprodukt der Zustände der einzelnen Qubits darstellen [14]. Die einzelnen Qubits besitzen dann keinen eigenen reinen Zustand mehr und können nur noch gemeinsam als zusammenhängendes System vollständig beschrieben werden. Eine unmittelbare Folge ist, dass die Messergebnisse verschränkter Qubits miteinander korreliert sind. Ein bekanntes Beispiel ist der Bell-Zustand

$$\left|\Phi^+\right\rangle = \frac{1}{\sqrt{2}}\left(|00\rangle + |11\rangle\right),$$

bei dem die Messung des einen Qubits das Ergebnis des anderen unmittelbar festlegt, obwohl vor der Messung keines der beiden einen definierten Wert besitzt. Diese nicht-klassischen Korrelationen sind eine wesentliche Ressource für viele Quantenalgorithmen und tragen maßgeblich zum Leistungsvorteil gegenüber klassischen Verfahren bei.

2.4. Bloch-Sphäre

Der Zustand eines einzelnen Qubits lässt sich geometrisch als Punkt auf der Oberfläche einer Einheitskugel darstellen, der sogenannten Bloch-Sphäre [14]. Da eine globale Phase physikalisch nicht messbar ist, genügen zwei reelle Winkel, um den Zustand vollständig zu beschreiben.

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + e^{i\varphi}\sin\left(\frac{\theta}{2}\right)|1\rangle\,.$$

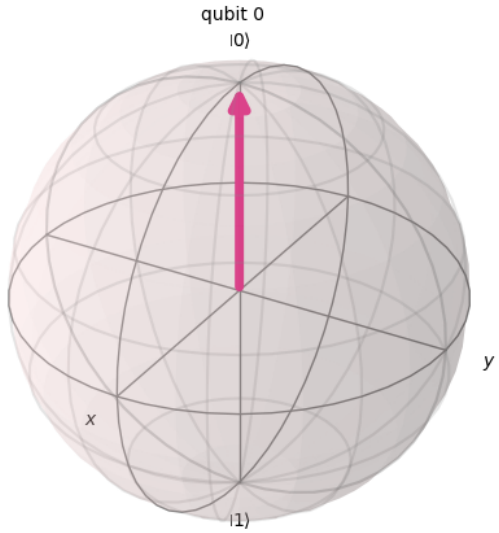


Abbildung 2.1.: Bloch-Sphäre im Basiszustand

Der Winkel θ bestimmt dabei das Verhältnis der beiden Amplituden, der Winkel φ ihre relative Phase. Die beiden Basiszustände $|0\rangle$ und $|1\rangle$ liegen am Nord- beziehungsweise Südpol der Kugel, während alle übrigen Punkte der Oberfläche Superpositionen entsprechen. Operationen auf einem Qubit lassen sich in dieser Darstellung als Drehungen des Zustandsvektors um die Achsen der Kugel auffassen. Die Bloch-Sphäre eignet sich damit vor allem zur Veranschaulichung einzelner Qubits.

2.5. Quantenschaltkreise und Gatter

Quantenschaltkreise, auch *Circuits* genannt, sind die übliche Darstellungsform für Quantenprogramme im Gattermodell, das auf David Deutsch [6] zurückgeht und heute den Standard für die Programmierung von Quantencomputern bildet. Jedes Qubit erhält seine eigene horizontale Linie, beginnend mit q_0.Bei n Qubits hat man daher n Linien, die von q_0 bis q_{n-1} nummeriert sind (siehe Abbildung 2.6).. Auf diese Linien werden von links nach rechts *Quantengatter* angewendet, die die Zustände der Qubits manipulieren [9], [14]. Am Ende eines Circuits steht in der Regel eine *Messung*, die den Quantenzustand auf einen klassischen Bitwert reduziert und pro Qubit den Wert 0 oder 1 liefert. Da das Ergebnis einer einzelnen Messung zufällig ist, lässt sich aus einem einzigen Durchlauf noch keine Aussage über den zugrundeliegenden Zustand ableiten. Erst die vielfache Ausführung desselben Circuits liefert eine Verteilung der Ergebnisse, aus der sich die Wahrscheinlichkeiten der möglichen Messwerte abschätzen lassen. Die Anzahl dieser Wiederholungen wird als *Shots* bezeichnet [1], [14].

Es gibt verschiedene Arten von Gattern. Einqubit-Gatter lassen sich als 2×2-Matrix darstellen. Jedoch gibt es auch Gatter, die auf mehrere Qubits wirken. Ein Gatter, das auf n Qubits wirkt, entspricht einer unitären $2^n \times 2^n$-Matrix.

X-Gatter

Das *X-Gatter* stellt das quantenmechanische Analogon der klassischen NOT-Operation dar und vertauscht die Amplituden der Basiszustände $|0\rangle$ und $|1\rangle$. Seine Matrixdarstellung lautet

$$\mathbf{X} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}. \tag{2.1}$$

Angewendet auf den Ausgangszustand $|0\rangle$ ergibt sich

$$\mathbf{X}\,|0\rangle = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix} = |1\rangle\,. \tag{2.2}$$

Der Zustand wird somit deterministisch in $|1\rangle$ überführt, es gilt also $p(0) = 0$ und $p(1) = 1$. Eine Superposition entsteht dabei nicht. Auf der Bloch-Sphäre entspricht das X-Gatter einer Drehung um π um die x-Achse, wodurch der Zustandsvektor vom Nordpol zum Südpol wandert (siehe Abbildung 2.2).



(a) Schaltkreis mit X-Gatter **(b)** Zustand auf der Bloch-Sphäre

Abbildung 2.2.: Wirkung des X-Gatters

Hadamard-Gatter

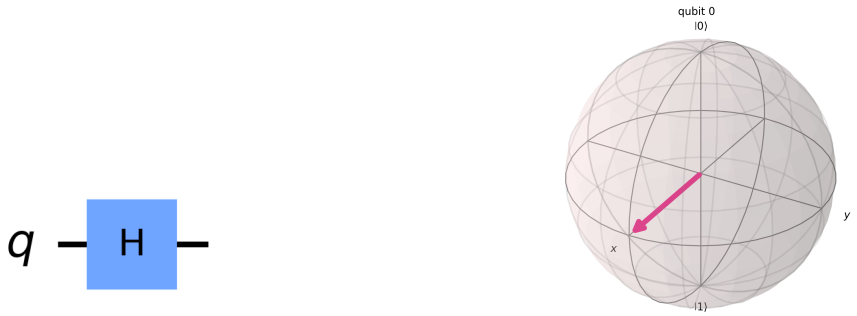
Das *Hadamard-Gatter* erzeugt aus einem Basiszustand eine gleichgewichtete Superposition und realisiert damit den Basiswechsel zwischen der Z- und der X-Basis. Es besitzt die Matrixdarstellung

$$\mathbf{H} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}. \tag{2.3}$$

Die Anwendung auf den Ausgangszustand $|0\rangle$ liefert

$$\mathbf{H}\,|0\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \frac{1}{\sqrt{2}}\Big(|0\rangle + |1\rangle\Big). \tag{2.4}$$

Beide Messergebnisse treten folglich mit gleicher Wahrscheinlichkeit $p(0) = p(1) = \frac{1}{2}$ auf, wie in Abbildung 2.3 dargestellt. Da $\mathbf{H}^2 = \mathbf{I}$ gilt, ist das Hadamard-Gatter zugleich sein eigenes Inverses.



(a) Schaltkreis mit H-Gatter **(b)** Zustand auf der Bloch-Sphäre

Abbildung 2.3.: Wirkung des Hadamard-Gatters

T-Gatter

Das *T-Gatter* gehört zur Klasse der Phasengatter. Es lässt den Basiszustand $|0\rangle$ unverändert und prägt dem Zustand $|1\rangle$ eine relative Phase von $e^{i\pi/4}$ auf. Seine Matrixdarstellung lautet

$$\mathbf{T} = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}. \tag{2.5}$$

Angewendet auf einen allgemeinen Zustand $|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$ ergibt sich

$$\mathbf{T}\,|\psi\rangle = \alpha\,|0\rangle + e^{i\pi/4}\beta\,|1\rangle\,. \tag{2.6}$$

Da die Phase den Betrag der Amplituden nicht verändert, bleiben die Messwahrscheinlichkeiten in der Rechenbasis unbeeinflusst. Die aufgeprägte Phaseninformation wird erst durch nachfolgende Gatter beobachtbar, die einen Basiswechsel bewirken.

Auf den Zustand $|0\rangle$ angewendet, bleibt das T-Gatter ohne Wirkung, da die Phase allein den Anteil $|1\rangle$ betrifft. Um die Phase überhaupt sichtbar zu machen, wird dem T-Gatter daher ein Hadamard-Gatter vorangestellt. Dieses erzeugt zunächst eine gleichgewichtete Superposition, sodass beide Basiszustände am Gesamtzustand beteiligt sind. Erst dann verschiebt das T-Gatter den Zustandsvektor auf der Bloch-Sphäre entlang des Äquators um 45 Grad, wie in Abbildung 2.4 dargestellt. Die Messwahrscheinlichkeiten bleiben dabei weiterhin bei $p(0) = p(1) = \frac{1}{2}$.



(a) Schaltkreis mit H- und T-Gatter **(b)** Zustand auf der Bloch-Sphäre

Abbildung 2.4.: Sichtbarwerden der Phase durch ein vorgeschaltetes Hadamard-Gatter

RY-Gatter

Das *RY-Gatter* gehört zur Klasse der Rotationsgatter und beschreibt eine Drehung des Qubitzustands um die y-Achse der Bloch-Sphäre um einen frei wählbaren Winkel θ. Seine Matrixdarstellung lautet

$$\mathbf{R}_y(\theta) = \begin{pmatrix} \cos\frac{\theta}{2} & -\sin\frac{\theta}{2} \\ \sin\frac{\theta}{2} & \cos\frac{\theta}{2} \end{pmatrix}. \tag{2.7}$$

Angewendet auf den Ausgangszustand $|0\rangle$ ergibt sich

$$\mathbf{R}_y(\theta)\,|0\rangle = \begin{pmatrix} \cos\frac{\theta}{2} & -\sin\frac{\theta}{2} \\ \sin\frac{\theta}{2} & \cos\frac{\theta}{2} \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \cos\tfrac{\theta}{2}\,|0\rangle + \sin\tfrac{\theta}{2}\,|1\rangle\,. \tag{2.8}$$

Der Winkel θ legt fest, wie stark sich der Zustand von $|0\rangle$ entfernt. Die Messwahrscheinlichkeiten betragen $p(0) = \cos^2\frac{\theta}{2}$ und $p(1) = \sin^2\frac{\theta}{2}$. Bei $\theta = 0$ bleibt $|0\rangle$ erhalten, bei $\theta = \pi$ entsteht $|1\rangle$. Dazwischen liegt eine Superposition beider Zustände. Für $\theta = \frac{\pi}{2}$ ergibt sich beispielsweise

$$\mathbf{R}_y\!\left(\tfrac{\pi}{2}\right)|0\rangle = \frac{1}{\sqrt{2}}\Big(|0\rangle + |1\rangle\Big), \tag{2.9}$$

also eine gleichgewichtete Superposition mit $p(0) = p(1) = \frac{1}{2}$, wie sie auch das Hadamard-Gatter aus $|0\rangle$ erzeugt (siehe Abbildung 2.5). Alle Einträge der Matrix sind reell, weshalb sich das RY-Gatter gut eignet, um klassische Daten in den Amplituden abzubilden.

Für beliebige Werte von θ gehört das RY-Gatter nicht zur Clifford-Gruppe. Als parametrisiertes Gatter wird es vor allem in variationellen Quantenschaltkreisen eingesetzt. Damit sind Schaltkreise gemeint, deren Parameter von einem klassischen Optimierer schrittweise angepasst werden, bis ein gewünschtes Ergebnis erreicht ist.



(a) Schaltkreis mit RY-Gatter **(b)** Zustand auf der Bloch-Sphäre

Abbildung 2.5.: Wirkung des RY-Gatters für $\theta = \frac{\pi}{2}$

CNOT-Gatter

Das *CNOT-Gatter* (auch *CX-Gatter*) ist das wichtigste Zweiqubit-Gatter. Es wirkt auf ein *Kontrollqubit* und ein *Zielqubit*. Befindet sich das Kontrollqubit im Zustand $|1\rangle$, wird auf das Zielqubit eine **X**-Operation angewendet, andernfalls bleibt das Zielqubit unverändert. In der Zweiqubit-Basis $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$ lautet die Matrixdarstellung

$$\mathbf{CX} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}. \tag{2.10}$$

Die Wirkung auf die Basiszustände ergibt sich entsprechend zu

$$\mathbf{CX}\,|00\rangle = |00\rangle, \quad \mathbf{CX}\,|01\rangle = |01\rangle, \quad \mathbf{CX}\,|10\rangle = |11\rangle, \quad \mathbf{CX}\,|11\rangle = |10\rangle. \tag{2.11}$$

Wird das Gatter auf einen Basiszustand angewendet, entsteht dabei stets wieder ein Produktzustand. Die besondere Bedeutung des CNOT-Gatters liegt darin, dass es aus einer Superposition des Kontrollqubits Verschränkung erzeugt.

$$\mathbf{CX}_{12}\,\mathbf{H}_1\,|00\rangle = \mathbf{CX}_{12}\,\frac{1}{\sqrt{2}}\Big(|00\rangle + |10\rangle\Big) = \frac{1}{\sqrt{2}}\Big(|00\rangle + |11\rangle\Big). \tag{2.12}$$

Das Hadamard-Gatter versetzt zunächst das erste Qubit in eine Superposition (siehe Abbildung 2.6). Das CNOT-Gatter wirkt anschließend auf beide Anteile zugleich. Der Anteil $|00\rangle$ bleibt unverändert, im Anteil $|10\rangle$ wird das Zielqubit umgeklappt. Im Ergebnis liefern beide Qubits bei einer Messung stets dasselbe Resultat, und der Zustand lässt sich nicht mehr als Tensorprodukt einzelner Qubitzustände schreiben.

Anders als bei den bisher betrachteten Gattern lässt sich das Ergebnis nicht mehr auf der Bloch-Sphäre darstellen. Diese beschreibt jeweils den Zustand eines einzelnen Qubits, ein verschränkter Zustand ist jedoch nur als Ganzes definiert. Betrachtet

man eines der beiden Qubits für sich, so liefert es in jeder Messrichtung beide Ergebnisse mit gleicher Wahrscheinlichkeit. Die eigentliche Information liegt nicht in den einzelnen Qubits, sondern in ihrer Korrelation und entzieht sich damit einer getrennten Darstellung.

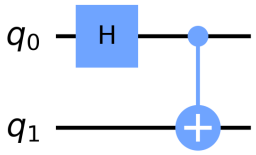


Abbildung 2.6.: Schaltkreis mit *H*- und *CNOT*-Gatter

2.6. Observablenberechnung

Als *Observable* wird in der Quantenmechanik eine messbare physikalische Größe bezeichnet, die durch einen hermiteschen Operator repräsentiert wird [14]. Ein Operator heißt *hermitesch*, wenn er mit seinem adjungierten Operator übereinstimmt, also $O = O^\dagger$ gilt, wobei $O^\dagger$ die konjugiert transponierte Matrix bezeichnet. Diese Eigenschaft stellt sicher, dass alle Eigenwerte reell sind, was Voraussetzung dafür ist, dass sie als physikalische Messwerte interpretiert werden können. Welcher Eigenwert bei einer einzelnen Messung auftritt, ist im Allgemeinen nicht vorhersagbar, sondern folgt einer durch den Quantenzustand festgelegten Wahrscheinlichkeitsverteilung.

Da einzelne Messungen zufällig ausfallen, ist im Quantencomputing weniger das Einzelergebnis als vielmehr der *Erwartungswert* einer Observablen O bezüglich eines Zustands $|\psi\rangle$ von Interesse. Dieser ist definiert als

$$\langle O \rangle = \langle \psi | \, O \, | \psi \rangle$$

und beschreibt den statistischen Mittelwert der Messergebnisse über viele Wiederholungen. Seine Berechnung beziehungsweise Schätzung bildet die Grundlage zahlreicher Quantenalgorithmen.

Für die konkrete Darstellung von Observablen werden üblicherweise die Pauli-Operatoren I, X, Y und Z herangezogen. Diese bilden mit reellen Koeffizienten eine Basis des Raums der hermiteschen 2×2-Matrizen, sodass sich jede Observable eines einzelnen Qubits als Linearkombination dieser vier Operatoren schreiben lässt [14]. Für Systeme mehrerer Qubits werden Observablen entsprechend als Linearkombination von Tensorprodukten dieser Operatoren ausgedrückt, die als „Pauli-Strings“ bezeichnet werden.

3. Quantencomputing-Software

Im Quantencomputing wird eine Vielzahl von Frameworks eingesetzt, die meist auf dem Konzept von Schaltkreisen und Gattern beruhen. In diesem Abschnitt werden die beiden Frameworks *PennyLane* und *Qiskit* vorgestellt, die später auch in der Implementierung und dem Benchmark genutzt werden. Zudem werden der Executor, in dem die Abstraktionsschicht implementiert wurde, sowie auf dem Markt vorhandene Alternativen vorgestellt.

3.1. PennyLane

PennyLane ist ein auf Python basierendes Open-Source-Framework (Apache License 2.0) für Quantencomputing, das von dem kanadischen Quantencomputing-Unternehmen Xanadu entwickelt wurde [3]. Das zentrale Merkmal des Frameworks ist die Berechnung von Gradienten parametrisierter Quantenschaltkreise [18] auf eine Weise, die mit klassischen Verfahren wie der Backpropagation kompatibel ist [2]. Die aus dem maschinellen Lernen bekannten Methoden der automatischen Differentiation werden so auf quantenmechanische und hybride Berechnungen ausgeweitet [3]. Damit ist *PennyLane* insbesondere auf den Einsatz von maschinellem Lernen mit Quantencomputern ausgelegt.

Von Haus aus liefert *PennyLane* eigene integrierte Simulatoren mit, darunter den Statevector-Simulator `lightning.qubit` [3]. Darüber hinaus stehen den Anwenderinnen und Anwendern über zahlreiche Plugins die Backends diverser externer Plattformen zur Verfügung, darunter IBM Quantum (*Qiskit*), Amazon Braket, Google *Cirq* sowie *Qulacs* [3]. In dieser Arbeit wird *PennyLane* in Version 0.42.3 verwendet.

3.2. Qiskit

Das bereits als Plugin-Backend erwähnte *Qiskit* ist ein auf Python basierendes Open-Source-Framework (Apache License 2.0) für Quantencomputing, das von IBM entwickelt und im Jahr 2017 veröffentlicht wurde [12]. Im Gegensatz zu *PennyLane*, dessen Architektur primär auf die differenzierbare Programmierung ausgerichtet ist, verfolgt *Qiskit* einen schaltkreiszentrierten Ansatz. Im Mittelpunkt steht die Klasse `QuantumCircuit`, mit der Quantenschaltkreise auf Gatterebene konstruiert, manipuliert und anschließend ausgeführt werden [12]. Ein zentrales Merkmal von *Qiskit* ist der mehrstufige Transpiler, der einen abstrakten Schaltkreis in eine für die Zielhardware ausführbare Form überführt [12].

Für die Simulation stellt das Zusatzpaket *Qiskit Aer* hochperformante, in C++ implementierte Simulatoren bereit, darunter einen Statevector-Simulator sowie Simulatoren auf Basis von Dichtematrizen und des Stabilizer-Formalismus [12]. Darüber hinaus ermöglicht *Qiskit* über das Paket `qiskit-ibm-runtime` den direkten Zugriff auf die Quantenprozessoren der IBM Quantum Platform [10], [11]. Durch Provider-Pakete von Drittanbietern lassen sich zudem Backends weiterer Hardware-Hersteller ansprechen [12]. In dieser Arbeit wird *Qiskit* in Version 2.4.1 verwendet.

3.3. Executor

Der *Executor* ist eine von Fraunhofer entwickelte Python-Bibliothek, mit der sich Quantenschaltkreise und Observablen über eine einheitliche Abstraktionsschicht definieren und anschließend in einem frei wählbaren Backend ausführen lassen. Die Software baut auf der ebenfalls von Fraunhofer entwickelten *sQUlearn*-Bibliothek auf, die intern ebenfalls einen Executor integriert. Die Grundidee besteht darin, die Beschreibung eines Quantenalgorithmus von der konkreten Software zu trennen, die ihn ausführt. Nutzerinnen und Nutzer formulieren einen Schaltkreis somit nur einmal und können ihn wahlweise etwa mit *Qiskit* oder *PennyLane* berechnen lassen, ohne den Code anpassen zu müssen. Das gewünschte Backend wird lediglich über seinen Namen ausgewählt.

Über diese Schnittstelle stehen die zentralen Operationen des Quantencomputing einheitlich zur Verfügung, insbesondere die Berechnung von Erwartungswerten, das Ziehen von Messproben (Sampling), die Bestimmung des Zustandsvektors sowie von Ableitungen. Die eigentliche Umrechnung in das jeweilige Zielformat übernimmt das Backend automatisch im Hintergrund, sodass sich die Nutzung für verschiedene

Frameworks gleich anfühlt. Dadurch verbessert die Bibliothek vor allem die Vergleichbarkeit und Reproduzierbarkeit von Ergebnissen über unterschiedliche Backends hinweg.

In dem Stand vor dieser Arbeit wurde für die Definition von Schaltkreisen und Observablen intern noch ein *Qiskit*-Wrapper verwendet. Die abstrakte Beschreibung stützte sich dabei auf *Qiskit*-Objekte als Zwischenrepräsentation und war somit noch an dieses Framework gebunden. Diese Abhängigkeit widerspricht dem Ziel einer vollständig framework-neutralen Abstraktionsschicht und wird in der Weiterentwicklung durch eine eigenständige, von *Qiskit* unabhängige Darstellung ersetzt. Im Verlauf dieser Arbeit wird gezeigt, wie in dem Executor eine solche Abstraktionsschicht entwickelt und implementiert wurde.

3.4. Stand der Technik

Einen vergleichbaren Ansatz zur *Executor Abstraction* wählt zudem die *qBraid-SDK*, mit der sich Quantenprogramme beliebiger populärer Quantenframeworks definieren lassen. Diese werden zunächst über einen graphbasierten Transpiler in ein vom Ziel-Backend erwartetes Format übersetzt und anschließend durch eine Kette von „Passes“ geleitet, die das Programm an die spezifischen Anforderungen der Zielhardware anpassen. Am Ende wird der Job über den „qBraid Runtime Service“ an das gewählte Backend übermittelt und dort berechnet. Ergänzend liefert die SDK Werkzeuge zur Nachbearbeitung und Visualisierung der Ergebnisse.

Im Gegensatz zum *Executor* liegt der Fokus der *qBraid-SDK* damit primär auf der einheitlichen Anbindung von Cloud-Quantenhardware und deren Simulatoren verschiedener Anbieter. Zu beachten ist zudem, dass der von qBraid empfohlene Workflow über den Runtime Service auf ein credit-basiertes Abrechnungsmodell zurückgreift, das sowohl den Zugriff auf QPUs als auch auf die verwalteten Cloud-Simulatoren umfasst.

Der *Executor* verfolgt hingegen das Ziel, ein einheitliches Interface für die Ausführung und vergleichende Bewertung verschiedener klassisch simulierbarer Backends bereitzustellen, um Benchmarking und Backend-Auswahl innerhalb einer einzigen Abstraktionsschicht zu ermöglichen. Da diese Backends lokal ausgeführt werden, entstehen für die Nutzenden keine laufenden Kosten für die Ausführung der Circuits.

4. Umsetzung und Implementierung der Abstraktionsschicht

In diesem Kapitel wird aufgezeigt, wie die Abstraktionsschicht aufgebaut und in den Executor integriert wurde. Hierbei wird zudem besprochen, welche Objekte zur Umwandlung genutzt wurden und weshalb.

4.1. Ausgangslage

Vor der Integration einer Definitionsschicht der Circuits gab es bereits die Basis der API-Anbindungen. Der damalige Ansatz bestand darin, einen Schaltkreis über *Qiskit*-Objekte zu definieren und anschließend an das Backend zu übergeben. Damit nutzte man jedoch keinen backendunabhängigen Ansatz, sondern lediglich einen Wrapper um *Qiskit*-Objekte. Jeder Schaltkreis war intern ein `qiskit.QuantumCircuit`, jede Observable eine `qiskit.SparsePauliOp` und jeder Parameter ein *Qiskit*-`ParameterVector` oder dessen Element. Wollte man ein anderes Backend als *Qiskit* verwenden, mussten diese *Qiskit*-Objekte zunächst in das jeweilige Backend-Format zurückübersetzt werden. Dadurch war das Programm ineffizient und von *Qiskit* abhängig.

4.2. Wahl der internen Objekte

Zunächst musste entschieden werden, in welches Format die Abstraktion die frameworkunabhängigen Objekte später umwandelt. Bei der Wahl der internen Objekte in *PennyLane* gab es zwei Optionen. In *PennyLane* unterscheidet man zwischen den Objekten „Quantum Tape“ und „Quantum Node“, um Circuits als Objekt zu erstellen.

Das Quantum Tape ist die unterste Abstraktionsschicht und wird hauptsächlich für die Entwicklung von Erweiterungen für *PennyLane* genutzt, da es einen kleineren

Overhead hat. Das Tape zeichnet die angewendeten Gatter als Liste von Operationen auf. Die Parameter der Gatter werden dabei als *PennyLane*-NumPy-Arrays gespeichert. Dies ist ein Wrapper der *NumPy*-Bibliothek, der die Parameter als differenzierbar markiert. Dadurch wird Backpropagation durch den Circuit möglich. Der Quantum Node ist eine Abstraktion des Quantum Tapes und die übliche Weise, einen Circuit zu erstellen. Dieses Objekt lässt sich durch den Python-Decorator `@qml.qnode` erstellen. Intern wird beim Aufruf des QNodes ein Tape erstellt und anschließend auf dem Device ausgeführt. Im Gegensatz zum Tape bietet der QNode mehrere Konfigurationsmöglichkeiten. Eine der wichtigsten hierbei ist die Option `diff_method="best"`, um eine alternative Differenzierungsmethode zu Parameter-Shift nutzen zu können. Funktional spricht dies für den QNode, hinsichtlich der Laufzeit war jedoch ein Vorteil der Tapes zu erwarten.

Um die beiden Objekte gegenüberzustellen, wurden zwei Vergleichsebenen betrachtet. Auf der ersten Ebene wurden Tape und QNode mit derselben Differenzierungsmethode (Parameter-Shift) betrieben. Damit lässt sich der reine Overhead isolieren, der durch die zusätzliche Abstraktionsschicht des QNodes entsteht, unabhängig davon, wie der Gradient berechnet wird. Auf der zweiten Ebene wurde der QNode zusätzlich mit der Option `diff_method="best"` gemessen. In diesem Modus wählt *PennyLane* auf dem Simulator `default.qubit` automatisch Backpropagation, die im Gegensatz zur Parameter-Shift-Regel nicht linear mit der Anzahl trainierbarer Parameter skaliert [13], [19]. Dieser Messpunkt bildet damit den Vorteil ab, den der QNode im praktischen Einsatz bietet, nämlich den Zugriff auf effizientere Differenzierungsmethoden ohne manuelle Implementierung der zugehörigen Ausführungslogik.

Die Messungen wurden für Circuits mit 5, 8, 10 und 12 Qubits durchgeführt und decken jeweils die Phasen Circuit-Erstellung, Ausführung und Gradientenberechnung bei 10 000 Gattern und 4 Iterationen ab. Bei der Gradientenberechnung besitzen 30 % der Gatter trainierbare Parameter, die über den Schaltkreis verteilt wurden. Die Ergebnisse sind in Abbildung 4.1 dargestellt.

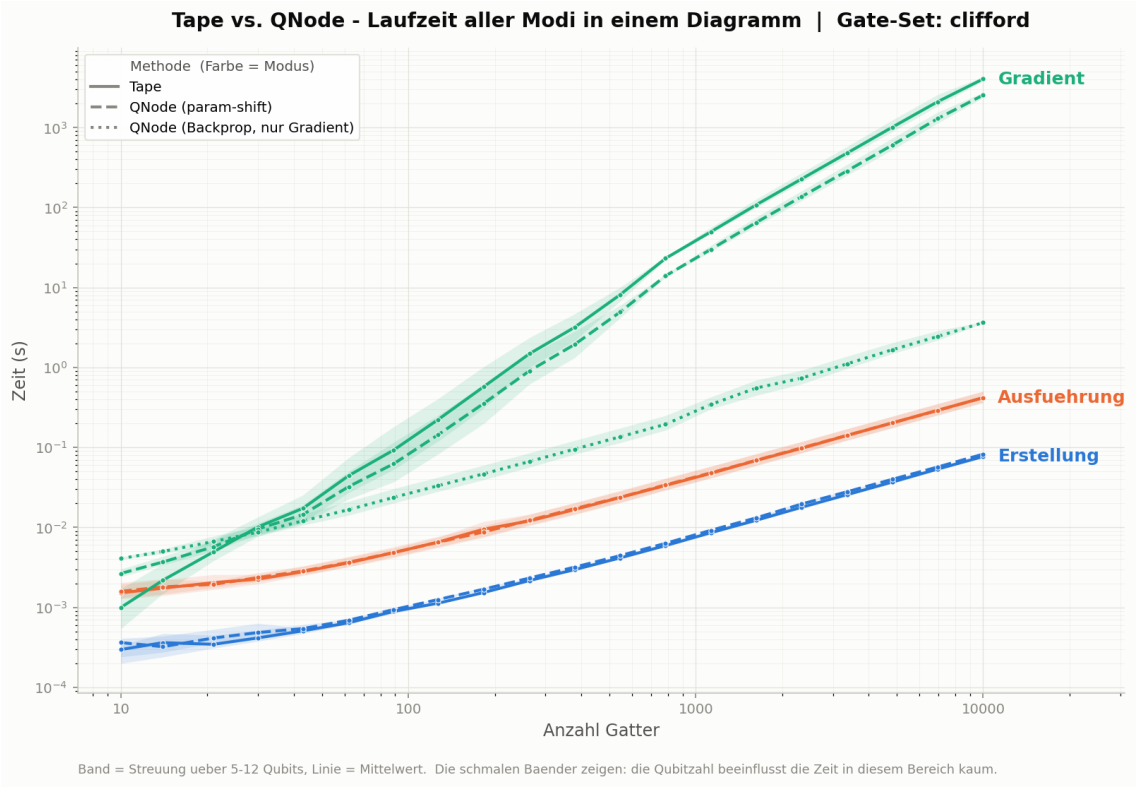


Abbildung 4.1.: Laufzeiten von Tape und QNode für Erstellung, Ausführung und Gradientenberechnung als Mittelwert von 5, 8, 10 und 12 Qubits

Es lässt sich erkennen, dass sowohl bei der Erstellung als auch bei der Ausführung und der Gradientenberechnung mittels Parameter-Shift kaum ein Unterschied zwischen den Tapes und den QNodes besteht. Zudem zeigt sich, dass die QNodes bei der Gradientenberechnung mit Backpropagation bei 10 000 Gattern einen Geschwindigkeitsvorteil von rund 36 % bis 38 % gegenüber den rohen Tapes haben. Parameter-Shift ist allgemein deutlich langsamer als die *best*-Methode (Backpropagation), da Parameter-Shift zwei Schaltkreis-Auswertungen pro trainierbarem Parameter benötigt (bei einer Anzahl von 3000 trainierbaren Gattern also $\approx 2 \cdot 3000 = 6000$ Auswertungen), während Backpropagation den vollständigen Gradienten in einem einzigen Vor- und Rückwärtslauf unabhängig von der Parameterzahl berechnet. Damit skaliert Parameter-Shift näherungsweise mit $\mathcal{O}(\text{Gatter}^2)$, Backpropagation dagegen mit $\mathcal{O}(\text{Gatter})$.

Ein weiteres wichtiges Argument für den QNode war, dass die Parameter der Gatter im Tape nur als feste Werte vorliegen und sich nicht als mathematische Ausdrücke angeben lassen. Da der QNode über eine Python-Funktion definiert wird, gehen die Parameter dort als Argumente ein und können in mathematischen Ausdrücken verwendet werden, bevor sie auf die Gatter angewendet werden. Das passende Tape entsteht daraus erst mit konkreten Werten.

Anders als in *PennyLane* gibt es in *Qiskit* nur ein Circuit-Objekt, den `QuantumCircuit`. Dieser entspricht dem Tape und war damit die einzige Wahl als internes Zielformat. Ein Gegenstück zum QNode gibt es nicht. Die Gradienten werden stattdessen über separate Parameter-Shift-Klassen aus `qiskit_algorithms.gradients` berechnet [19].

4.3. Implementierung der Abstraktion

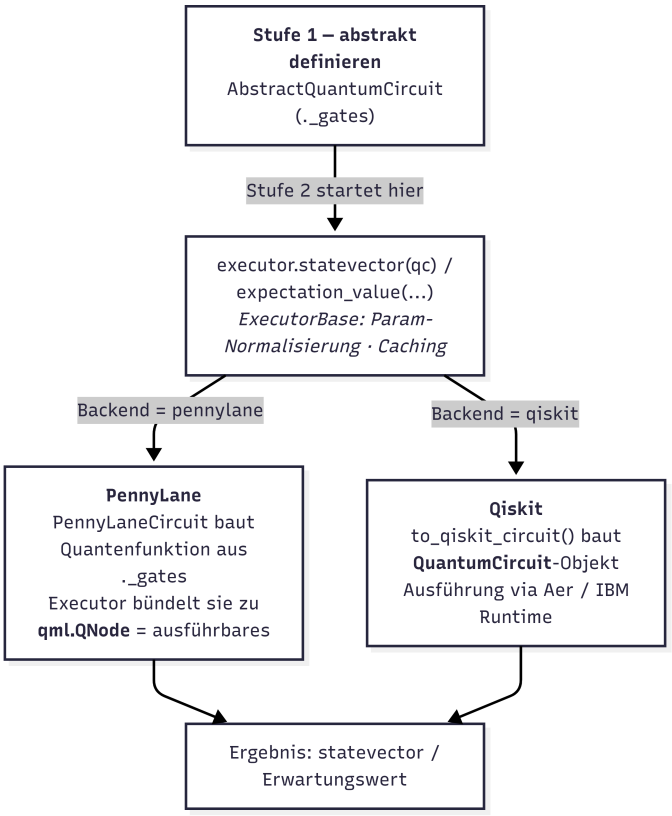


Abbildung 4.2.: Architektur der Abstraktion

Grundlage der Abstraktionsschicht ist eine Datenstruktur, mit der sich ein Schaltkreis und Observablen über eine eigene, vom Framework unabhängige Syntax beschreiben lassen. Die wesentliche Schwierigkeit bestand darin, den gemeinsamen Funktionsumfang der unterstützten Frameworks zu ermitteln, da nur solche Konstrukte in die Abstraktion aufgenommen werden können, die sich in jedem Ziel-Backend nachbilden lassen.

Die Abstraktionsschicht ruht auf vier Datentypen, aus denen jedes Backend anschließend seine eigenen, nativen Objekte erzeugt. Entscheidend ist, dass diese vier Typen frameworkunabhängig sind. Sie beschreiben, *was* ausgeführt und gemessen werden soll, treffen jedoch keinerlei Annahmen über das ausführende Framework und importieren insbesondere keines davon. Damit wird die im Konzept geforderte Trennung von Definition und Ausführung (Abschnitt **??**) auf der Ebene der Datentypen konkret. Die Nutzerinnen und Nutzer definieren ihren Schaltkreis einmalig in dieser abstrakten Form, während die Übersetzung in ein konkretes Framework erst später und lokal im jeweiligen Backend erfolgt. Hierbei lässt sich das Framework durch das Ändern einer einzigen Variablen wechseln.

Der Schaltkreis

Ein Schaltkreis wird framework-unabhängig als geordnete Liste typisierter Gate-Objekte gehalten. Die Reihenfolge der Listenelemente bestimmt die Ausführungsreihenfolge. Aus dieser Repräsentation ergeben sich zwei Eigenschaften. Zum einen stellen alle Schaltkreis-Operationen wie das Verketten, Invertieren, Kopieren und das Binden von Parametern Umformungen dieser Liste dar und erfordern keinen Aufruf einer Framework-Funktion. Zum anderen gelangt jedes Gatter über eine einzige Stelle in die Liste, an der die Validierung erfolgt. Geprüft werden die Zugehörigkeit zum gemeinsamen Vokabular, die erwartete Form des Gatters hinsichtlich der Zahl seiner Qubits und Winkel sowie die Einhaltung der Registergröße. Nicht unterstützte Konstrukte werden dadurch bereits bei der Definition zurückgewiesen und nicht erst beim Aufbau des nativen Schaltkreises, bei dem sich der Ort der Fehlermeldung deutlich von der Ursache entfernen würde.

Das Gate-Vokabular

Ein einzelnes Gatter wird framework-unabhängig als *Dataclass* abgebildet. Die Abstraktion unterscheidet dabei lediglich zwei Formen, ein `CliffordGate` für parameterfreie Gatter wie `h` oder `cx` und ein `RotationGate` für einwinklige Gatter wie `rx` oder `rzz`. Die Beschränkung auf zwei Formen folgt aus der Anforderung, dass jedes Backend das Gatter ohne vorherige Zerlegung abbilden können muss.

Die zulässigen Gatternamen werden ausschließlich durch die Registry `SUPPORTED_GATES` festgelegt, die in der umgesetzten Fassung 33 Gatter umfasst. Sie dient als einzige verbindliche Referenz, aus der sowohl die Validierung als auch die Übersetzer beider Backends lesen. Ein Eintrag ist erst dann gültig, wenn er an drei Stellen zugleich verfügbar ist, und zwar als Methode auf *Qiskit*s `QuantumCircuit`, als Schlüssel im Übersetzungswörterbuch des *PennyLane*-Backends sowie, sofern das Gatter parameterfrei ist, in den Inversionstabellen der Abstraktion. Rotationen werden stattdessen über das Vorzeichen ihres Winkels invertiert.

Da die Konsistenz dieser drei Stellen technisch nicht erzwungen wird, sichert ein eigener Paritätstest sie ab. Dieser beschränkt sich nicht auf die Prüfung der Existenz, sondern vergleicht die von beiden Backends erzeugten Unitärmatrizen und verifiziert, dass Gatter und Inverses gemeinsam die Identität ergeben. Ein neu aufgenommenes Gatter ohne angebundenes Backend führt daher zu einem Fehlschlag der Testsuite und nicht zu einem Fehler zur Laufzeit.

Die Observable

Das Gegenstück zum Schaltkreis beschreibt die zu messende Größe, also das

`observable`-Argument von `expectation_value`. Dieses akzeptiert wahlweise eine einzelne Observable oder eine Liste davon, sodass mehrere Erwartungswerte in einem Aufruf ausgewertet werden können. Die Speicherung erfolgt ebenfalls rein datenbasiert als Liste von Pauli-Strings mit zugehörigen Koeffizienten. Beispielsweise beschreibt `["ZZ", "IX"]` mit `[0.5, 0.3]` den Operator $0{,}5\,Z_0Z_1 + 0{,}3\,X_0$. Da die Observable framework-unabhängig bleibt, umfasst die Komponente eine eigene Pauli-Algebra. Die Verknüpfung zweier Operatoren sowie das Zusammenfassen gleichartiger Terme sind auf Basis einer Produkttabelle der einzelnen Pauli-Zeichen eigenständig implementiert. Festgelegt ist zudem eine Little-Endian-Konvention, bei der das rechte Zeichen Qubit 0 bezeichnet. Eine derartige Festlegung ist einmalig zu treffen und anschließend in jedem Übersetzer zu berücksichtigen. *PennyLane* ordnet Pauli-Zeichen dagegen von links nach rechts zu, was als Big-Endian bezeichnet wird, weshalb das entsprechende Backend das Label vor der Übergabe umkehrt. Bei der Umwandlung für das *Qiskit*-Backend ist dies nicht nötig, da *Qiskit* bereits Little-Endian nutzt.

Die Parameter

Ein Parameter stellt einen symbolischen Platzhalter für einen Zahlenwert dar und ist als Unterklasse von `sympy.Symbol` realisiert. Jeder Parameter trägt den Namen seines Vektors sowie einen Index und wird als `theta[0]` notiert. Die Symbole werden zudem auf die reelle Domäne eingeschränkt, was der projektweiten Annahme ausschließlich reeller Parameter entspricht.

Aus dieser Realisierung ergeben sich zwei Eigenschaften. Erstens sind Parameter vollwertige *SymPy*-Symbole, sodass ein Ausdruck wie `2*theta[0] + theta[1]` unmittelbar gültig ist, ohne dass eine eigene Arithmetik oder ein eigener Ausdrucksbaum implementiert werden müsste. Die symbolische Auswertung übernimmt *SymPy*. Zweitens enthält ein solcher Ausdruck ausschließlich Daten und keinerlei Framework-Bezug. Derselbe Ausdruck kann daher an beiden Stellen verwendet werden, an denen die Abstraktion einen Zahlenwert erwartet, nämlich als Gate-Winkel und als Operator-Koeffizient.

Der `ParameterVector` fasst Parameter zu einer benannten, indexierten und in ihrer Länge veränderbaren Folge zusammen. Die in einem Schaltkreis oder Operator enthaltenen Parameter werden stets nach Vektorname und Index sortiert zurückgegeben. Diese deterministische Ordnung ist keine bloße Formalie, sondern definiert die Reihenfolge, in der die Backends die Parameterwerte zur Ausführungszeit erwarten.

Die Übersetzung in den Parametertyp des jeweiligen Frameworks übernimmt das Backend. In beiden Fällen wird hierfür *SymPy*s `lambdify` eingesetzt, allerdings zu unterschiedlichen Zwecken. Für *Qiskit* wird der Ausdruck auf die Elemente eines

Qiskit-`ParameterVector` angewandt und dadurch als *Qiskit*-`ParameterExpression` rekonstruiert, bleibt also symbolisch. Für *PennyLane* wird der Ausdruck hingegen in eine numerisch auswertbare Funktion überführt, die zur Ausführungszeit mit den konkreten Parameterwerten aufgerufen wird. Ein und derselbe Mechanismus bedient somit sowohl die symbolische als auch die numerische Zielrepräsentation.

4.4. Integration in den Executor

Die Abstraktion beschreibt bisher nur die Definition eines Schaltkreises und einer Observable. Ausgeführt wird beides über den bereits vorhandenen Executor, weshalb die Abstraktion an drei Stellen in die bestehende Basis eingefügt werden musste, und zwar in die Basisklassen, in die Backends und in die Auswahl des Backends.

Die Basisklassen `QuantumCircuitBase` und `QuantumOperatorBase` legen fest, welche Methoden ein Schaltkreis und eine Observable bereitstellen müssen. Bisher waren sie auf *Qiskit* typisiert, da ihre Winkel und Parameter als *Qiskit*-Objekte angegeben waren. Diese Angaben wurden entfernt. Ob ein Winkel umgerechnet oder unverändert an das Gatter durchgereicht wird, entscheidet sich nun daran, ob es sich um eine Zahl handelt, und nicht mehr am Typ des Objekts. Dadurch arbeiten die zusammengesetzten Methoden der Basisklasse wie `pauli_evolution` mit den symbolischen Parametern der Abstraktion genauso wie zuvor mit denen von *Qiskit*. Die Abstraktion erbt selbst von diesen Basisklassen und ist damit kein zusätzliches Format neben dem bestehenden, sondern dessen Umsetzung.

Die Übersetzung in das native Format geschieht erst im Backend. Jedes Backend besitzt dafür einen Eintrittspunkt, der aus dem abstrakten Objekt das native erzeugt. Entscheidend ist dabei weniger, was dieser annimmt, als was er ablehnt. Übergeben werden dürfen nur Objekte der Abstraktion, ein `qiskit.QuantumCircuit` wird zurückgewiesen. Damit ist der eingangs beschriebene Weg über *Qiskit*-Objekte geschlossen, diese sind nur noch das Ergebnis der Übersetzung und nicht mehr deren Eingabe. Blieben beide Wege offen, bestünde die Abhängigkeit von *Qiskit* weiterhin. Aus demselben Grund prüft auch das *Qiskit*-Backend jeden Gatternamen gegen `SUPPORTED_GATES`, obwohl *Qiskit* das Gatter selbst auflösen könnte. Andernfalls ließen sich Schaltkreise definieren, die nur auf *Qiskit* lauffähig sind. Das Ergebnis der Übersetzung wird über den Hash des abstrakten Objekts zwischengespeichert und nur nach einer Änderung neu erzeugt.

Am Executor selbst waren nur wenige Ergänzungen nötig. Eine übergebene Observable wird vor der Ausführung selbsttätig in den nativen Typ gewandelt, sodass die

Abstraktion direkt an `expectation_value` übergeben werden kann. Die Parameterwerte werden weiterhin über den Namen des Vektors angegeben, etwa `theta=[0.3, 0.7]`. Welcher Wert bei welchem Symbol landet, entscheidet die weiter oben beschriebene Sortierung nach Vektorname und Index, an der sich beide Backends beim Binden orientieren. Eine falsche Zuordnung würde keinen Fehler auslösen, sondern lediglich ein falsches Ergebnis liefern.

Welches Backend verwendet wird, entscheidet die Factory über `Executor.create()`. Sie hält eine Registry aus Backend-Namen und den zugehörigen Klassen, in die sich ein Backend beim Import selbst einträgt. Die Definition von Schaltkreis und Observable bleibt beim Wechsel unverändert, geändert wird allein der Name des Backends.

Zuletzt wurden die alten Wrapper entfernt, in denen Schaltkreis, Observable und Parameter als *Qiskit*-Objekte gehalten wurden. Dadurch importieren weder die Basisklassen noch die Abstraktion noch das *PennyLane*-Backend *Qiskit*. Die Installation des *PennyLane*-Backends zieht *Qiskit* nicht mehr nach, was zuvor nicht möglich war, da die Übersetzung nach *PennyLane* über *Qiskit*-Objekte lief.

5. Vorteile der Abstraktionsschicht

Die in Kapitel 4 beschriebene Abstraktionsschicht adressiert vier wiederkehrende Probleme bei der Arbeit mit mehreren Quantencomputing-Frameworks. Dazu zählen die Bindung des Programmcodes an eine konkrete Framework-Syntax, die eingeschränkte Vergleichbarkeit von Implementierungen, der Installationsaufwand paralleler Framework-Stacks und die Instabilität der Framework-APIs über Versionsgrenzen hinweg. Die folgenden Abschnitte behandeln diese Punkte einzeln. Abschnitt 5.5 grenzt anschließend ab, welche Zusagen die Abstraktion nicht macht.

5.1. Einheitliche Definitionsschnittstelle

Der zentrale Effekt der Abstraktion besteht darin, dass für die Beschreibung eines Schaltkreises und einer Observablen genau ein Weg existiert. Ein Schaltkreis wird ausschließlich über `AbstractQuantumCircuit` definiert, eine Observable ausschließlich über `AbstractQuantumOperator`. Welche Gatter zulässig sind, ergibt sich aus einem einzigen, explizit gepflegten Vokabular in `SUPPORTED_GATES`. Jedes hinzugefügte Gatter wird unmittelbar dagegen validiert.

Ein Wechsel des Ausführungsbackends erfordert dadurch keine Änderung der Problembeschreibung. Ein direkt gegen ein Framework geschriebenes Programm muss bei einem Wechsel vollständig auf die Syntax und die Objektmodelle des Zielframeworks portiert werden. Hier beschränkt sich der Eingriff auf den Austausch der Executor-Instanz.

```
# Schaltkreisdefinition, backendunabhängig
qc = AbstractQuantumCircuit(2)
qc.h(0)
qc.cx(0, 1)
# Auswahl des Backends
executor = Executor.create("qiskit")
# executor = Executor.create("pennylane")
```

Neben der Ersparnis an Portierungsaufwand ist vor allem die methodische Konsequenz relevant. Da beide Ausführungen aus derselben Schaltkreisdefinition hervorgehen, ist die Formulierung des Problems über alle Backends hinweg eine kontrollierte Größe. Beobachtete Unterschiede in Laufzeit oder Ergebnis lassen sich damit auf das Backend zurückführen und nicht auf eine abweichende Formulierung des Schaltkreises. Diese Eigenschaft ist die Voraussetzung für die in Kapitel **??** durchgeführten Vergleichsmessungen. Ohne sie wäre ein Backend-Vergleich stets mit dem Implementierungsstil der jeweiligen Referenzimplementierung vermischt.

Die zentrale Validierung verlagert Fehler zudem nach vorn. Ein unbekannter Gatter-Name oder eine falsche Qubit-Anzahl führt bereits beim Aufbau des Schaltkreises zu einer Ausnahme. Andernfalls würde der Fehler erst bei der Ausführung im Backend auftreten und dort nur in framework-spezifischer Terminologie sichtbar.

5.2. Modularisierbarkeit

Die Abstraktion zerlegt ein Quantenprogramm in austauschbare Bestandteile. Dies betrifft sowohl die Trennung von Definition und Ausführung als auch den Aufbau eines Schaltkreises aus einzelnen Teilen.

Trennung von Definition und Ausführung

Die Beschreibung des Problems ist vollständig von seiner Ausführung getrennt. Das Paket `abstraction` enthält keinerlei Framework-Importe. Die Übersetzung in ein natives Format findet erst zur Laufzeit innerhalb des jeweiligen Backends statt. Der Nutzercode hängt damit von den abstrakten Verträgen in `base` ab und nicht von einer konkreten Implementierung. Dies entspricht dem Dependency-Inversion-Prinzip **martin2003agile**. Ein Programm ist dadurch kein „*Qiskit*-Programm“ oder „*PennyLane*-Programm“ mehr, sondern ein Programm, das auf *Qiskit* oder *PennyLane* ausgeführt wird. Die Methode `switch_backend` überträgt zusätzlich die bestehende Konfiguration auf ein anderes Backend.

Zusammensetzen von Teilschaltkreisen

Ein Schaltkreis liegt intern als typisierte Liste von Gatter-Objekten vor und nicht als framework-spezifische Datenstruktur. Er lässt sich dadurch wie ein wiederverwendbarer Baustein behandeln. `compose` setzt einen Teilschaltkreis an einer frei wählbaren Qubit-Position ein und macht ihn damit ortsunabhängig. `invert` erzeugt

das zugehörige Gegenstück und `copy` stellt sicher, dass sich mehrfach verwendete Bausteine nicht gegenseitig beeinflussen. Variationsalgorithmen lassen sich so in getrennt entwickelbare und einzeln testbare Blöcke gliedern, etwa in Zustandspräparation, Ansatz und Basiswechsel.

Späte Parameterbindung

Symbolische Parameter beschreiben einen Schaltkreis, dessen Zahlenwerte noch offen sind. Sie werden erst bei der Ausführung übergeben, sodass derselbe Schaltkreis mit beliebig vielen Parametersätzen wiederverwendet werden kann. Für einen klassischen Optimierer genügt damit der Zugriff auf die Parameter und den zugehörigen Erwartungswert. Der innere Aufbau des Schaltkreises bleibt ihm verborgen, wodurch sich Optimierung und Schaltkreisdefinition unabhängig voneinander entwickeln lassen.

Erweiterbarkeit

Die Backends sind der Abstraktion nicht namentlich bekannt. Sie registrieren sich über eine Registry und werden zusätzlich über Entry-Points der Gruppe `qc_executor.backends` entdeckt. Ein zusätzliches Backend erweitert das System, ohne dass Änderungen am Kern erforderlich sind. Dies entspricht dem Open-Closed-Prinzip.

5.3. Optionale Framework-Abhängigkeiten

Da die Abstraktion selbst kein Framework importiert, muss auch nicht die Gesamtheit der unterstützten Frameworks installiert sein. Die Backends sind als optionale Installationsvarianten deklariert und werden beim Import in einem `try/except ImportError`-Block geladen. Ein nicht installiertes Framework deaktiviert ausschließlich das zugehörige Backend, während alle übrigen Backends unverändert nutzbar bleiben. Der Versuch, ein nicht verfügbares Backend zu erzeugen, führt zu einer Fehlermeldung mit dem passenden Installationshinweis.

Praktisch ist dies aus zwei Gründen relevant. Zum einen reduziert es den Umfang einer Installation auf die tatsächlich benötigten Komponenten, was insbesondere in Container- und CI-Umgebungen ins Gewicht fällt. Zum anderen entschärft es Abhängigkeitskonflikte. Die unterstützten Frameworks geben teilweise eigene und gegenseitig unverträgliche Versionsbereiche für gemeinsame Basisbibliotheken vor. Da sich nur eine Teilmenge der Backends installieren lässt, addieren sich diese Einschränkungen nicht zu einem unauflösbaren Gesamtsystem.

5.4. Entkopplung von Framework-Versionen

Die APIs der betrachteten Frameworks unterliegen einer erheblichen Änderungsrate. Hauptversionswechsel gehen regelmäßig mit dem Entfernen oder Umbenennen öffentlicher Schnittstellen einher. Ohne Abstraktionsschicht wirkt sich eine solche Änderung unmittelbar auf jedes Anwendungsprogramm aus, das die betroffene Schnittstelle verwendet.

Die Abstraktion begrenzt die Reichweite derartiger Änderungen strukturell. Da Framework-Aufrufe ausschließlich innerhalb der Backend-Module auftreten, endet die Auswirkung einer API-Änderung an der Modulgrenze. Zusätzlich sind Versionsfallunterscheidungen in einem eigenen Kompatibilitätsmodul zusammengefasst. Dieses stellt Versionsflags und Wrapper um geänderte Schnittstellen zentral bereit, sodass der übrige Code frei von versionsabhängigen Verzweigungen bleibt.

Ein Beispiel ist der Zugriff auf den symbolischen Ausdruck eines *Qiskit*-Parameters. Dieser erfolgte bis Version 2.0 über ein privates Attribut und seither über eine öffentliche Methode. Die Anpassung betrifft eine einzige Funktion und nicht jede aufrufende Stelle.

Hierdurch entsteht keine Unabhängigkeit von Framework-Änderungen, sondern eine Lokalisierung des Anpassungsaufwands. Der Aufwand verschwindet nicht, sondern verlagert sich von den Anwendungsprogrammen in eine klar umrissene und zentral testbare Schicht.

Für Anwendende bedeutet dies, dass ein einmal geschriebenes Programm über Framework-Versionswechsel hinweg unverändert lauffähig bleibt. Für die Wartung der Bibliothek bedeutet es, dass Anpassungen an genau einer Stelle erfolgen und durch die Testsuite abgesichert sind, insbesondere durch den Paritätstest über alle Backends.

5.5. Einschränkungen

Die genannten Vorteile werden durch Einschränkungen erkauft, die für eine belastbare Bewertung zu benennen sind. Zunächst ist die Ausdrucksstärke der Schnittstelle begrenzt, da sie exakt durch das gemeinsame Gatter-Vokabular definiert ist. Framework-spezifische Funktionalität wie Transpiler-Optionen, Rauschmodelle oder Hardware-Topologien liegt außerhalb der Abstraktion und ist über sie nicht erreichbar. Die Vereinheitlichung erfolgt notwendigerweise über der Schnittmenge der Frameworks und nicht über ihrer Vereinigung. Hinzu kommt der Übersetzungsaufwand, denn die Umwandlung in das native Format verursacht zusätzliche Laufzeit gegenüber einer direkten Implementierung. Der Umfang dieses Mehraufwands wird im folgenden Kapitel 6 quantifiziert.

6. Evaluation und Ergebnisse

In diesem Kapitel wird ein Benchmark durchgeführt, der die Performance des Executors in Bezug auf die Laufzeit den nativen Frameworks gegenüberstellt. Verglichen wird dabei die benötigte Zeit für die Erstellung, die Ausführung sowie die Gradientenberechnung der Quantenschaltkreise. Zunächst wird der experimentelle Aufbau der Messungen detailliert beschrieben. Anschließend werden die Ergebnisse anhand von Diagrammen visuell aufbereitet und diskutiert. Ziel dieser Untersuchung ist es, die Laufzeiteffizienz des Executors zu bestimmen und den zeitlichen Overhead gegenüber der direkten Nutzung der nativen Frameworks zu quantifizieren.

6.1. Aufbau der Messung

Gemessen wird nicht die Leistung der Simulatoren selbst, sondern der Aufwand, den die Abstraktionsschicht zusätzlich verursacht. Da im Messfenster jeweils der gesamte Aufruf einschließlich der Simulation liegt, lässt sich dieser Aufwand nur über einen Vergleich bestimmen. Jede Messung wurde deshalb zweimal durchgeführt, einmal direkt über die native API des Frameworks und einmal über den Executor mit demselben Framework als Backend. Die erste Variante wird im Folgenden als nativer Benchmark bezeichnet. Da beide Seiten denselben Simulator mit demselben Schaltkreis aufrufen, kürzt sich der Simulationsanteil im Verhältnis der beiden Laufzeiten weitgehend heraus. Dieses Verhältnis wird als Overhead-Faktor angegeben.

Damit wirklich derselbe Schaltkreis ausgeführt wird, entsteht die Gattersequenz zunächst framework-unabhängig aus einem festen Seed und wird erst danach in beide Darstellungen übersetzt. Genutzt wurde ein Set aus 13 Clifford- und Nicht-Clifford-Gattern. Die Gatterzahl läuft über 20 logarithmisch verteilte Werte zwischen 10 und 100 000, die Qubit-Zahl über 5, 8, 10 und 12. Jede Zelle wurde viermal mit unterschiedlichen Zufallsschaltkreisen gleicher Größe wiederholt.

Als Messgröße dient die Wall-Clock-Zeit über `time.perf_counter()`. Vor jeder Messung läuft ein ungemessener Aufruf, damit Lazy-Imports und interne Caches nicht in die Messung fallen. Im Messfenster liegt ausschließlich der jeweils untersuchte Aufruf,

während die Erzeugung des Devices, der Aufbau des Circuits und die Instanziierung des Estimators zur ungemessenen Setup-Phase gehören.

Gemessen wurde in drei Modi, die typische Phasen eines variationellen Workflows abbilden. Der Modus `creation` baut nur die Circuit-Repräsentation des jeweiligen Frameworks oder Executors auf, der Modus `execution` wertet den Erwartungswert $\langle Z_0 \rangle$ aus und der Modus `gradient` bildet dessen analytische Ableitung. Für den Gradienten wurde jedes dritte Gatter durch eine trainierbare Rotation ersetzt, sodass die Gatterzahl konstant bleibt, während die Zahl der Ableitungsrichtungen mit $n_{\text{trainable}} = \lceil n_{\text{gates}}/3 \rceil$ mitwächst.

6.2. PennyLane und Executor im Vergleich

Für den Vergleich wird die native *PennyLane*-QNode dem Executor auf dem *PennyLane*-Backend gegenübergestellt. Das Ergebnis ist auf beiden Seiten dasselbe, nur der Weg dorthin unterscheidet sich. Die native Implementierung spricht *PennyLane* unmittelbar an, der Executor dagegen erst über die abstrakte Zwischenschicht. Was an Zeit zwischen beiden liegt, geht damit auf die Abstraktion zurück.

Alle übrigen Bedingungen wurden konstant gehalten. Als Simulator dient in beiden Fällen `qml.device("default.qubit")` im analytischen Modus mit `shots=None`, wobei die native Implementierung die Wire-Zahl vorgibt und der Executor sie offen lässt. Ein Ergebnis-Cache kommt weder bei der QNode über `cache=False` noch beim Executor über den Standardwert `caching=False` zum Einsatz. Die Gattersequenz stammt aus demselben Generator mit demselben Seed, sodass jeweils derselbe Schaltkreis entsteht. Auch die Observable stimmt überein, denn der Executor übersetzt `AbstractQuantumOperator(paulis=["I...IZ"], coeffs=[1.0])` nach `qml.expval(1.0 * Z(0))` und damit bis auf den skalaren Faktor nach `qml.expval(qml.PauliZ(0))`. Der Messablauf ist ebenfalls identisch, mit vier Wiederholungen, einem frischen Schaltkreis je Wiederholung und einem ungemessenen Warm-up-Aufruf vorab. Der Aufbau von Gerät, QNode und Executor bleibt auf beiden Seiten außerhalb des Messfensters.

Dadurch fällt in jedem der drei Modi auf beiden Seiten dieselbe Arbeit an, und die Differenz spiegelt allein den Aufwand der Abstraktion wider. Was das im Einzelnen bedeutet, zeigen die folgenden Absätze.

Modus `creation`

Die native Implementierung misst `qml.workflow.construct_tape(circuit)()`. Dieser Aufruf läuft einmal durch die Quantenfunktion und trägt sämtliche Operationen in ein `QuantumTape` ein, ohne das Gerät anzusprechen oder zu simulieren.

Beim Executor wird `ex.transpile_circuit(build_abstract(...))` gemessen. Zunächst entsteht die abstrakte Gatterliste, anschließend deren Übersetzung ins native Format, also Listen aus *PennyLane*-Operationen, Wires und Winkeln. Auch hier bleibt es beim reinen Aufbau ohne Tape und ohne Simulation, weshalb die Differenz ausschließlich aus dem Erstellen und Übersetzen der Gatterliste besteht.

Modus `execution`

Die native Implementierung misst `circuit()`. Der Aufruf zeichnet die Quantenfunktion auf, reicht das Tape an das Preprocessing des Geräts weiter, simuliert und liefert $\langle Z_0 \rangle$.

Beim Executor wird `ex.expectation_value(qc_abs, op)` gemessen, wobei Schaltkreis und Observable zu diesem Zeitpunkt bereits vorliegen. Ins Messfenster fallen die Normalisierung der Parameter, der Cache-Lookup des Circuits, der Aufbau einer neuen QNode sowie die Simulation. Für den Lookup wird `hash(AbstractQuantumCircuit)` gebildet, was über sämtliche Gatter läuft. Neu eingereiht werden die Operationen bei jedem Aufruf ohnehin auf beiden Seiten, auf der nativen Seite wegen `cache=False` und beim Executor wegen der frisch gebauten QNode. Als Differenz verbleibt somit die zusätzliche Python-Schicht.

Modus `gradient`

Die native Implementierung misst `qml.grad(circuit)(params)` mit `params = pnp.array(np.zeros(n_trainable), requires_grad=True)` Die QNode trägt `diff_method="best"`,sodass *PennyLane* auf `default.qubit` zu Backpropagation greift, wie in Abschnitt 4.2 dargelegt. Beim Executor wird `ex.expectation_value_derivatives(qc_abs, op, "x", x=values)` gemessen. Er erzeugt dazu eine `qml.QNode(circuit_func, device, diff_method="best", max_diff=1)`, kennzeichnet die Werte über `requires_grad=True` und differenziert mit `qml.jacobian`. Da die QNode einen Skalar zurückgibt, entspricht das Ergebnis dem von `qml.grad`.

6.3. Qiskit und Executor im Vergleich

Die zweite Paarung stellt die native *Qiskit*-API dem Executor auf dem *Qiskit*-Backend gegenüber. Der Aufbau folgt demselben Prinzip wie in Abschnitt 6.2, nur dass diesmal beide Seiten *Qiskit* ansprechen, die eine unmittelbar und die andere über die abstrakte Zwischenschicht.

Alle übrigen Bedingungen wurden konstant gehalten. Gerechnet wird in beiden Fällen über dasselbe Primitiv, den `StatevectorEstimator` im exakten Modus mit `shots=None`. Die native Implementierung legt ihn selbst an, während `Executor.create("qiskit")` ihn intern über den Standard-Backend-String `"statevector"` erzeugt. Ein Ergebnis-Cache kommt nicht zum Einsatz, da `caching` beim Executor standardmäßig ausgeschaltet ist. Die Gattersequenz stammt aus demselben Generator mit demselben Seed, sodass jeweils derselbe Schaltkreis entsteht. Auch die Observable stimmt überein. Die native Implementierung nutzt `SparsePauliOp.from_sparse_list([("Z", [0], 1.0)], num_qubits=n)` und der Executor übersetzt `AbstractQuantumOperator(paulis=["I...IZ"], coeffs=[1.0])` in genau denselben `SparsePauliOp`. Der Messablauf ist ebenfalls identisch, mit vier Wiederholungen, einem frischen Schaltkreis je Wiederholung und einem ungemessenen Warm-up-Aufruf vorab. Circuit, Observable und Primitiv werden auf beiden Seiten außerhalb des Messfensters aufgebaut.

Zu begründen ist dabei, weshalb die native Implementierung über den Estimator misst und nicht unmittelbar über `Statevector`. Intern rechnet der Executor stets über `OpTree` und den *Qiskit*-Estimator; einen Weg ohne Estimator gibt es dort nicht, denn die Ableitung bricht sonst mit einem `RuntimeError` ab. Würde die native Implementierung direkt über `Statevector` rechnen, steckte in der Differenz zusätzlich der Primitiv-Overhead von *Qiskit* selbst und nicht mehr allein der der Abstraktion. Hinzu kommt, dass der Estimator das Gegenstück zur QNode von *PennyLane* bildet, da beide Circuit und Observable entgegennehmen und einen Erwartungswert liefern. Nativer und Executor-Benchmark liegen damit in beiden Frameworks auf derselben Ebene.

Auch hier leisten damit beide Seiten in jedem der drei Modi dieselbe Arbeit. Die folgenden Absätze halten fest, worin sie sich je Modus unterscheiden.

Modus `creation`

Die native Implementierung misst das Anlegen und Befüllen eines `QuantumCircuit(n)`. Die Gatter werden über die üblichen Methoden angehängt, also `qc.h`, `qc.cx`, `qc.rx`

und weitere. Ein Primitiv wird dabei nicht herangezogen und es wird nichts ausgeführt.

Beim Executor wird `ex.transpile_circuit(build_abstract(...))` gemessen. Zunächst entsteht die abstrakte Gatterliste, anschließend deren Übersetzung in einen nativen `QuantumCircuit`. Jedes Gatter wird beim Anhängen gegen das gemeinsame Vokabular `SUPPORTED_GATES` geprüft. Da kein Backend gesetzt ist, folgt darauf keine Transpilation, sodass im Messfenster die Übersetzung selbst liegt und kein *Qiskit*-Transpiler-Pass. Am Ende steht auf beiden Seiten derselbe gefüllte native `QuantumCircuit`, ohne dass simuliert wurde. Anders als bei *PennyLane* braucht es dafür keinen Umweg, denn *Qiskit* baut den Schaltkreis unmittelbar auf, statt ihn erst beim Ausführen aufzuzeichnen. Die Differenz umfasst damit den Aufbau der abstrakten Gatterliste, deren Prüfung und deren Übersetzung.

Modus `execution`

Die native Implementierung misst `estimator.run([(qc, observable)]).result()`. Schaltkreis, Observable und Estimator stehen zu diesem Zeitpunkt bereits fest, sodass nur die Auswertung ins Messfenster fällt.

Beim Executor wird `ex.expectation_value(qc_abs, op)` gemessen. Ins Messfenster fallen die Normalisierung der Parameter, das Nachschlagen des bereits übersetzten Schaltkreises, das Verpacken in `OpTree`-Knoten, der Aufbau der Parameter-Dictionaries und schließlich derselbe Estimator-Aufruf. Die Übersetzung selbst fällt nur einmal an, da sie am abstrakten Circuit zwischengespeichert wird. Als Schlüssel dient `hash(AbstractQuantumCircuit)`, und dieser Hash läuft bei jedem Aufruf erneut über sämtliche Gatter. Da auf beiden Seiten dieselbe Simulation desselben Schaltkreises mit derselben Observable über dasselbe Primitiv abläuft, verbleibt als Differenz die `OpTree`-Schicht, das Hashing für den Cache-Lookup und die Parameterverwaltung.

Modus `gradient`

Auf der nativen Seite wird der Gradient über eine manuelle Umsetzung der Parameter-Shift-Regel berechnet. Für jeden Parameter θ_k werden dazu zwei Aufrufe an den `Estimator` abgesetzt, einmal mit dem Wert $\theta_k + \pi/2$ und einmal mit $\theta_k - \pi/2$, woraus sich der Gradient gemäß der Formel

$$\mathrm{grad}_k = \frac{f(\theta_k + \pi/2) - f(\theta_k - \pi/2)}{2}$$

ergibt. Die Parameterwerte werden dabei zur Laufzeit direkt im Primitiv gebunden, indem sie als dritter Eintrag des sogenannten PUB (Primitive Unified Bloc) übergeben werden, anstatt sie vorab über `assign_parameters` fest in den Schaltkreis einzusetzen.

Beim Executor wird `ex.expectation_value_derivatives(qc_abs, op, "x", x=values)` gemessen. Er baut über `OpTreeDerivative.differentiate` je Parameter den Parameter-Shift-Ausdruck auf und wertet ihn über `OpTreeEvaluate` mit demselben Estimator aus.

Eine Besonderheit ist an dieser Stelle zu erläutern. Anders als bei *PennyLane* wird der Schaltkreis auf beiden Seiten einmalig im ungemessenen Aufbau erzeugt, sodass allein die Ableitung im Messfenster liegt. Bei *PennyLane* zeichnet die native Implementierung bei jedem Aufruf neu auf, weshalb auch der Executor jedes Mal neu übersetzen muss. Bei *Qiskit* bleibt der `QuantumCircuit` dagegen bestehen und lediglich die Werte wechseln, sodass der Executor seinen Schaltkreis ebenfalls behalten darf.

Ausgewertet wird auf der nativen Seite bewusst sequenziell, ohne die Verschiebungen zu einem Aufruf zusammenzufassen, da auch der Executor Parameter für Parameter auswertet. Ein solches Batching spart gemessen ohnehin nur etwa sechs Prozent. Zu beachten ist außerdem der Aufwand, denn der Parameter-Shift braucht zwei Ausführungen pro Parameter und die Parameterzahl wächst mit der Gatterzahl. Je Messaufruf steigt der Aufwand dadurch etwa quadratisch mit der Gatterzahl. Das betrifft beide Seiten gleichermaßen, macht den Modus aber teuer.

6.4. Ergebnisse des Vergleichs von PennyLane und Executor

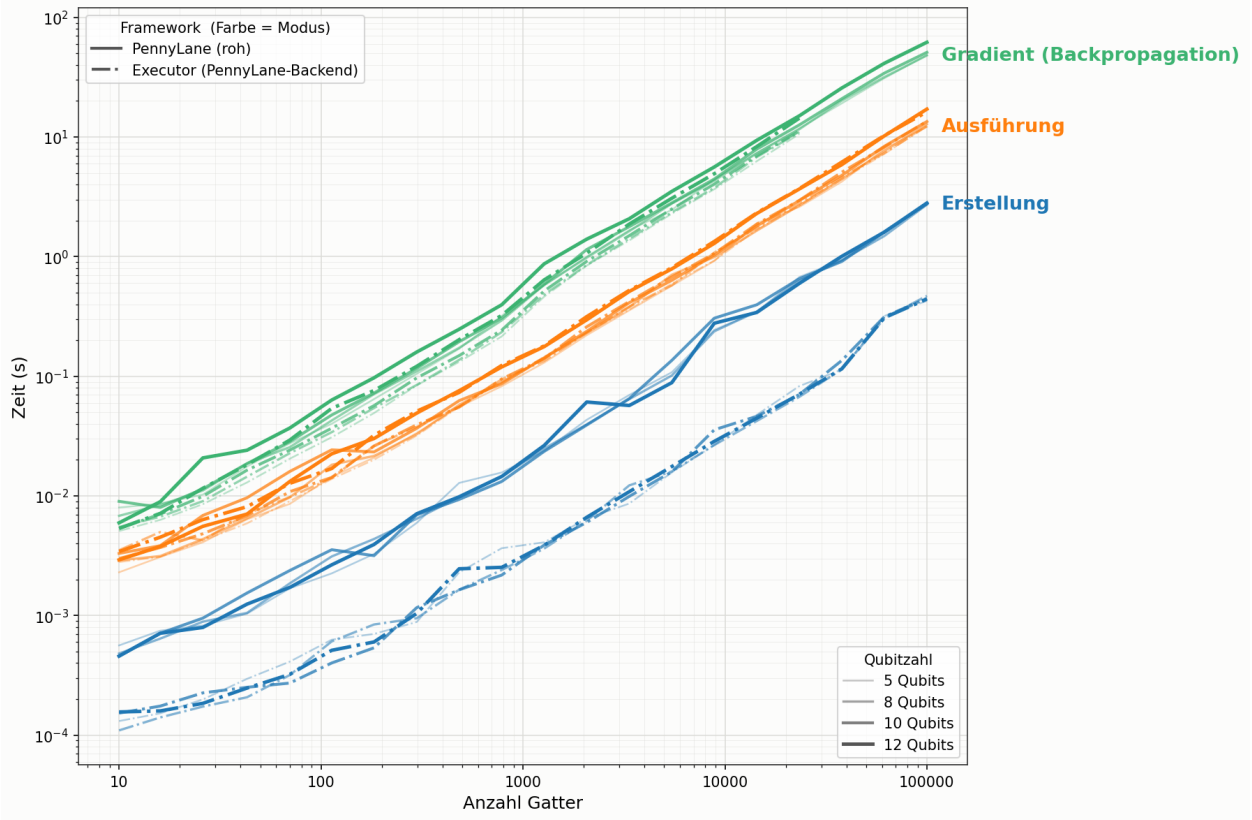


Abbildung 6.1.: Laufzeit von nativem *PennyLane* (durchgezogen) und Executor mit *PennyLane*-Backend (strichpunktiert)

Gemessen wurde über die Qubit-Zahlen 5, 8, 10 und 12 sowie über 20 logarithmisch verteilte Gatterzahlen zwischen 10 und 100 000, jede Zelle mit vier Wiederholungen. Abbildung 6.1 zeigt das Ergebnis doppelt-logarithmisch für alle drei Modi.

Die Qubit-Zahl macht in diesem Bereich nur einen minimalen Unterschied. Die vier Linien einer Kombination aus Modus und Framework fallen so eng zusammen, dass sie als ein einziges Band erscheinen. Bei 100 000 Gattern steigt die native Laufzeit im Modus `creation` von 2,76 s bei 5 Qubits auf 2,80 s bei 12 Qubits, im Modus `execution` von 12,1 s auf 17,1 s. Das ist ein Faktor von 1,03 beziehungsweise 1,41, während sich der Hilbertraum zwischen beiden Konfigurationen um den Faktor 128 vergrößert. Die Laufzeit wird folglich nicht von der Zustandsvektor-Arithmetik dominiert, sondern von der pro Gatter anfallenden Python-Arbeit. Nur unter dieser Voraussetzung wird der Overhead einer zusätzlichen Abstraktionsschicht überhaupt sichtbar; für deutlich größere Register ist zu erwarten, dass er relativ zur Simulation verschwindet.

Die Reihen laufen im Modus `creation` und `execution` auf beiden Seiten bis 100 000 Gatter durch. Im Modus `gradient` reicht die Executor-Seite aus Laufzeitgründen nur bis 23 357 Gatter, die native Seite bis 100 000. Alle Gradienten-Aussagen beziehen sich daher auf den gemeinsamen Bereich bis 23 357 Gatter.

Als Kennzahl dient im Folgenden das geometrische Mittel von $t_{\text{Executor}}/t_{\text{nativ}}$ über alle Messpunkte ab 1000 Gattern und alle vier Qubit-Zahlen. Unterhalb von 1000 Gattern liegen die Laufzeiten im Bereich weniger Millisekunden, wo Einmaleffekte die Verhältnisse verrauschen.

Erstellung mit erwartbarem Ergebnis

Im Modus `creation` liegt der Faktor bei 0,14, der Executor ist dort also rund siebenmal schneller als die native API. Bei 5 Qubits und 100 000 Gattern stehen 0,43 s gegen 2,76 s. Über die vier Qubit-Zahlen ist der Wert praktisch konstant (0,141 bis 0,146), und die Steigungen im Log-Log-Bild sind mit 1,07 bis 1,10 auf beiden Seiten gleich. Es handelt sich also um einen reinen Vorfaktor und nicht um einen zusätzlichen Skalierungsterm.

Der Grund liegt darin, dass über die Abstraktionsschicht gar kein vollständiges Tape entsteht. `ex.transpile_circuit(...)` übersetzt die abstrakte Gatterliste lediglich in drei parallele Python-Listen mit der *PennyLane*-Operationsklasse, der Wire-Liste und dem Winkel je Gatter. Die native Referenz `qml.workflow.construct_tape (circuit)()` durchläuft demgegenüber die gesamte Quantenfunktion, instanziiert für jedes Gatter ein Operationsobjekt, reiht dieses über den Konstruktor in die gerade aktive Queue ein und setzt daraus ein ausführungsfertiges `QuantumTape` zusammen. Tracing, Queuing und Objekterzeugung entfallen beim Executor vollständig.

Diese Zahl ist damit die am wenigsten belastbare der drei und sollte nicht als Effizienzgewinn gelesen werden, denn beide Seiten leisten hier nicht dieselbe Arbeit. Das Instanziieren der Operationen holt der Executor bei jeder Ausführung nach. Der Vorsprung ist keine eingesparte, sondern eine verschobene Arbeit. Genau deshalb schlägt er im Modus `execution` nicht mehr durch.

Ausführung ohne messbaren Overhead

Im Modus `execution` liegen beide Seiten dicht aufeinander; die strichpunktierten Executor-Linien liegen innerhalb der Streuung auf den durchgezogenen nativen Linien. Das geometrische Mittel beträgt 1,011, der Executor kostet also rund 1 % mehr Laufzeit. Je Qubit-Zahl streut der Wert zwischen 0,99 und 1,03, in Einzelpunkten zwischen 0,82 und 1,12, und das Vorzeichen wechselt. Bei 8 und 12 Qubits ist der Executor am rechten Rand sogar geringfügig schneller (12,31 s gegen 12,80 s

beziehungsweise 16,21 s gegen 17,10 s). Ein Overhead von einem Prozent liegt deutlich unterhalb dessen, was diese Messanordnung auflösen kann; die belastbare Aussage lautet daher, dass gegenüber *PennyLane* kein messbarer Overhead entsteht, und nicht, dass er exakt 1,1 % beträgt.

Erklärbar ist das über denselben Mechanismus wie bei der Erstellung. Der Executor spielt bei jedem Aufruf eine vorbereitete Instruktionsliste in einer Python-Schleife ab, während der native Benchmark die Quantenfunktion erneut durchläuft und jedes Gatter über die Konstruktoren der Operationsklassen in das Tape einreiht. Die zusätzliche Ebene des Executors ersetzt hier einen mindestens ebenso teuren nativen Pfad, statt ihn zu ergänzen.

Gradientenberechnung mit unerwartetem Ergebnis

Im Modus `gradient` fällt das Ergebnis unerwartet aus. Das geometrische Mittel liegt bei 0,86, der Executor ist dort also um 12 bis 15 % *schneller* als die native API, und zwar durchgängig über alle vier Qubit-Zahlen (0,853 bis 0,878). Bei 5 Qubits und 23 357 Gattern stehen 10,60 s beim Executor gegen 11,83 s auf der nativen Seite. Das kann so nicht stimmen. Der Executor leistet dieselbe Arbeit wie der native Benchmark, denn Gerät und Schaltkreis sind identisch, und über `diff_method="best"` rechnet auch er auf `default.qubit` mit Backpropagation. Zusätzlich übersetzt er aus der abstrakten Darstellung. Die Abstraktionsschicht müsste einen Overhead mit sich bringen, kann den nativen Pfad aber nicht unterbieten.

Unabhängig von der Höhe des Faktors ist das Skalierungsverhalten unauffällig. Beide Seiten wachsen linear in der Gatterzahl, die Steigungen liegen nativ bei 0,99 bis 1,02 und beim Executor bei 1,05 bis 1,07. Das bestätigt die Eigenschaft von Backpropagation, dass ein einzelner Rückwärtsdurchlauf alle Ableitungen liefert und seine Kosten nur von der Schaltkreisgröße abhängen, nicht von der Parameterzahl; gegenüber der reinen Ausführung kostet der Gradient lediglich einen konstanten Faktor von etwa 4. Bemerkenswert ist die kleine, aber über alle vier Qubit-Zahlen gleichgerichtete Differenz von rund 0,05 im Exponenten. Der Vorsprung des Executors schrumpft zum rechten Rand hin, von 0,86 im Mittel auf 0,89 bis 0,96 am letzten gemeinsamen Messpunkt. Der Effekt ist also nicht konstant, sondern zehrt sich mit wachsender Schaltkreisgröße auf.

Diskussion des Gradienten-Ergebnisses

Als Ursache für den Wert 0,86 kommt in erster Linie die Laufzeitumgebung in Betracht, denn die Executor-Gradientenreihe stammt nicht aus demselben Lauf wie die übrigen Messungen. Der Docker-Container, in dem die Benchmarks ausgeführt

werden, wurde während des ursprünglichen Laufs abgebrochen; die Gradientenreihe des Executors wurde deshalb am 31. 07. 2026 als eigener Lauf nachgezogen, während alle übrigen Reihen beider Seiten vom 27. 07. 2026 stammen. Zwischen beiden Zeitpunkten wurde das Container-Image neu gebaut, und dabei änderte sich die *PennyLane*-Version. In der Abhängigkeitsliste war *PennyLane* zunächst ohne Versionsangabe geführt, sodass die nativen Reihen gegen die zum Build-Zeitpunkt jeweils neueste Version liefen; für den nachgezogenen Gradientenlauf des Executors war die Abhängigkeit dagegen auf *PennyLane* 0.42.3 festgeschrieben. Die beiden Seiten des Gradienten-Vergleichs wurden also nicht gegen dieselbe Bibliotheksversion gemessen. Da ein wesentlicher Teil der gemessenen Zeit in *PennyLane*s eigener Python-Schicht anfällt, also in Tracing, Queuing und Preprocessing des Tapes, schlagen sich Änderungen an dieser Schicht unmittelbar in der Laufzeit nieder, und zwar nur auf einer der beiden Seiten des Vergleichs. Wie groß dieser Beitrag ist, lässt sich nachträglich nicht quantifizieren, da für den früheren Lauf nur die Abhängigkeit ohne Versionsangabe dokumentiert ist und nicht die tatsächlich installierte Version.

6.5. Ergebnisse des Vergleichs von Qiskit und Executor

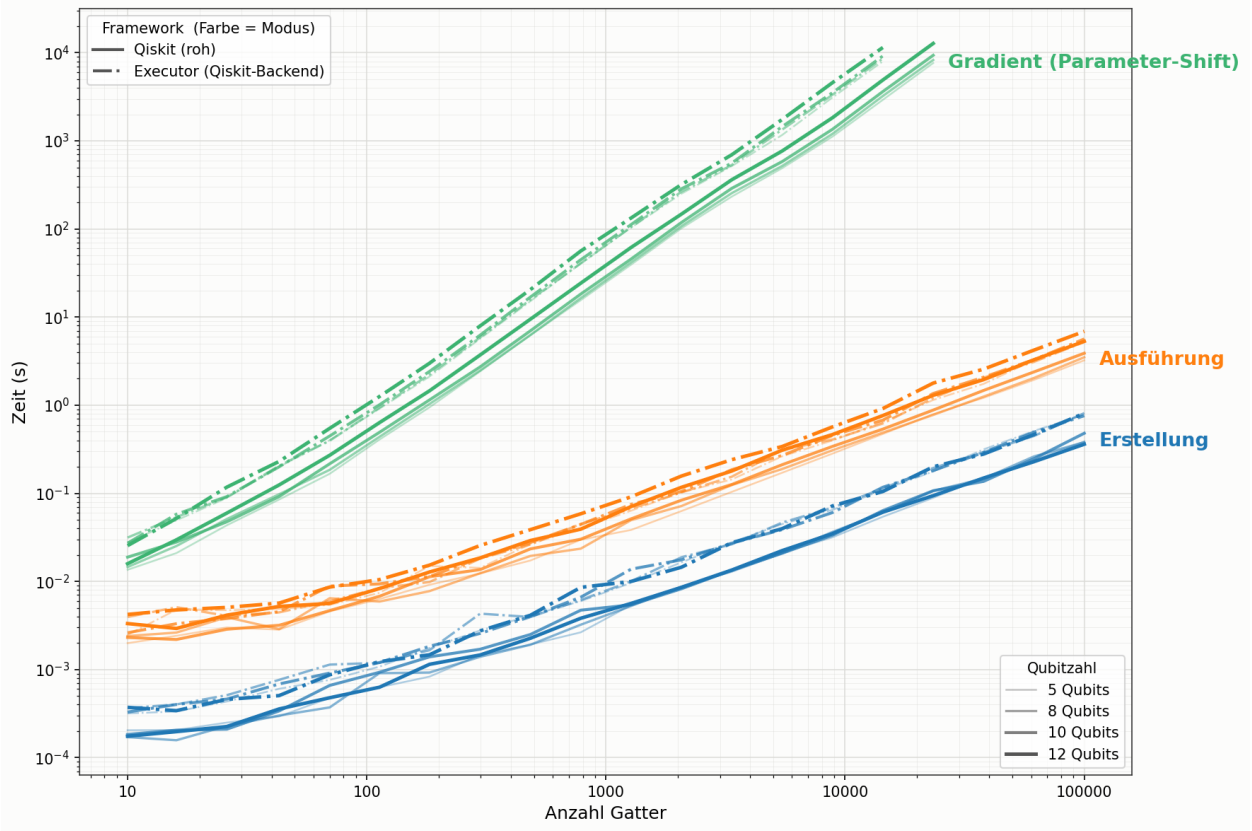


Abbildung 6.2.: Laufzeit von nativem *Qiskit* (durchgezogen) und Executor mit *Qiskit*-Backend (strichpunktiert)

Für das *Qiskit*-Paar gelten dasselbe Messraster und dieselbe Kennzahl wie oben. Abbildung 6.2 zeigt das Ergebnis in derselben Auftragung. Das Bild unterscheidet sich in einem Punkt grundlegend von Abbildung 6.1. Die strichpunktierten Executor-Linien liegen in allen drei Modi sichtbar oberhalb der durchgezogenen nativen Linien. Gegenüber *Qiskit* kostet die Abstraktion also messbar.

Auch hier macht die Qubit-Zahl nur einen minimalen Unterschied, und zwar aus demselben Grund wie zuvor. Im Modus `creation` liegen die nativen Zeiten bei 100 000 Gattern ohne erkennbaren Trend zwischen 0,36 s und 0,48 s, im Modus `execution` steigen sie von 3,23 s bei 5 Qubits auf 5,31 s bei 12 Qubits. Das ist ein Faktor von 1,64 bei einem 128-fach größeren Hilbertraum.

Die Gradientenreihe reicht nativ bis 23 357 Gatter und auf der Executor-Seite bis 14 384; die folgenden Gradienten-Aussagen beziehen sich auf den gemeinsamen Bereich bis 14 384 Gatter. Der Grund für den früheren Abbruch ist das Laufzeitbudget, denn eine einzelne Zelle kostet dort bereits mehr als zwei Stunden.

Erstellung mit konstantem Faktor 2

Im Modus `creation` liegt das Verhältnis bei 1,97, die Einzelwerte je Qubit-Zahl streuen nur zwischen 1,93 und 2,02. Die Steigungen sind mit 0,96 bis 1,02 auf beiden Seiten praktisch identisch, der Overhead ist also ein reiner Vorfaktor und kein zusätzlicher Skalierungsterm. Bei 5 Qubits und 100 000 Gattern stehen 0,79 s beim Executor gegen 0,39 s nativ.

Anders als bei *PennyLane* ist dieser Wert unmittelbar interpretierbar, denn beide Seiten liefern am Ende dieselbe Datenstruktur, nämlich einen vollständig befüllten nativen `QuantumCircuit`. Es wird nichts in eine spätere Phase verschoben. Der Executor leistet davor zusätzlich die Arbeit der abstrakten Ebene aus Prüfung jedes Gatters gegen das gemeinsame Vokabular, Aufbau einer Parameter-Map, Nachschlag der passenden Circuit-Methode und Hashing für die Memoisierung. Genau diese Zusatzarbeit ist der Faktor 2.

Ausführung mit Faktor 1,4 und abnehmender Tendenz

Über alle Qubit-Zahlen gemittelt liegt der Overhead bei 1,40. Hier zeigt die Qubit-Achse als einzige im gesamten Datensatz einen klaren, monotonen Trend. Der Faktor sinkt von 1,51 bei 5 Qubits über 1,44 und 1,38 auf 1,28 bei 12 Qubits.

Die Erklärung liegt im Nenner und nicht im Zähler. Die absolute Differenz zwischen beiden Seiten bleibt bei 100 000 Gattern mit 2,07 s, 1,73 s, 1,76 s und 1,58 s über die vier Qubit-Zahlen praktisch konstant, während die native Laufzeit im selben Schritt von 3,23 s auf 5,31 s wächst. Der Zusatzaufwand des Executors aus Aufbau und Auswertung der OpTree-Struktur, Deduplizierung und Parameter-Bookkeeping ist reine Python-Buchhaltung ohne Bezug zur Registergröße. Der relative Overhead wird also amortisiert, sobald die Simulation teurer wird, und für große Register ist zu erwarten, dass er weiter gegen 1 läuft.

Gradientenberechnung mit erwartetem Vorzeichen, aber ohne konstanten Faktor

Der Gradienten-Modus ist der teuerste Punkt der Paarung mit einem Mittel von 2,40 und Einzelwerten von 2,53 bei 5 Qubits bis 2,22 bei 12 Qubits. Anders als bei *PennyLane* fällt das Ergebnis hier in die erwartete Richtung aus. Der Executor ist langsamer als die native API, so wie es eine zusätzliche Abstraktionsschicht sein muss. In absoluten Zahlen dauert ein Gradient bei 5 Qubits und 14 384 Gattern nativ 2922 s und über den Executor 7985 s, was einer Differenz von rund 84 Minuten pro Aufruf entspricht.

Entscheidend ist, dass dieser Faktor als einziger im gesamten Datensatz mit der Gatterzahl wächst. Über die untere Hälfte des Messbereichs liegt er bei 1,91 bis 2,25, über die obere bei 2,23 bis 2,53 und am letzten gemeinsamen Messpunkt bei 2,34 bis 2,73. Sichtbar wird das an den Steigungen im gemeinsamen Bereich, die nativ bei 1,74 bis 1,77 und beim Executor bei 1,78 bis 1,82 liegen. Die Differenz von rund 0,03 bis 0,04 im Exponenten ist klein, aber über alle vier Qubit-Zahlen gleichgerichtet und damit kein Rauschen. Sie passt zu den zwei tiefen Circuit-Kopien, die der Executor für die Parameter-Shift-Regel je Parameter anlegt. Jede einzelne kostet $\mathcal{O}(n_{\text{gates}})$ und fällt für $\mathcal{O}(n_{\text{gates}})$ Parameter an. Zusammen tragen sie damit einen eigenen quadratischen Term bei, statt den vorhandenen nur zu skalieren.

Dass die gemessenen Exponenten mit etwa 1,75 unterhalb des theoretischen Werts von 2 liegen, widerspricht dieser Abschätzung nicht. Die Zahl der Estimator-Läufe wächst exakt linear mit der Gatterzahl, die Kosten eines einzelnen Laufs steigen im unteren Teil des Messbereichs aber noch unterlinear, weil dort Fixkosten des Primitivs mitlaufen. Die Steigung nähert sich am rechten Rand entsprechend an 2 an.

Auf die Einschränkung aus Abschnitt 6.4 ist diese Paarung nicht angewiesen, denn beide *Qiskit*-Reihen stammen aus demselben Lauf vom 27. 07. 2026 und damit aus demselben Container-Image mit denselben Bibliotheksversionen. Der Wechsel der *PennyLane*-Version betrifft sie nicht, und die drei Faktoren sind unabhängig davon interpretierbar.

7. Fazit

7.1. Zusammenfassung

Ziel dieser Arbeit war es, eine framework-unabhängige Abstraktionsschicht für die Definition parametrisierter Quantenschaltkreise und Observablen zu entwerfen, im *Executor* zu implementieren und ihren Laufzeit-Overhead gegenüber der direkten Nutzung der Frameworks zu bestimmen. Dieses Ziel wurde erreicht.

Kapitel 4 beantwortet FF1. Die Abstraktion ruht auf vier framework-unabhängigen Datentypen, deren zulässiges Vokabular zentral in `SUPPORTED_GATES` festgelegt und durch einen Paritätstest über beide Backends abgesichert ist. Die bisherige Bindung an *Qiskit* wurde damit aufgelöst, da weder die Basisklassen noch das *PennyLane*-Backend *Qiskit* importieren.

Kapitel 5 beantwortet FF2. Als qualitative Vorteile wurden eine einheitliche Definitionsschnittstelle, die Modularisierbarkeit von Schaltkreisen, optionale Framework-Abhängigkeiten sowie die Entkopplung von Framework-Versionen herausgearbeitet. Zugleich wurde gezeigt, dass sich die Ausdrucksstärke der Schnittstelle notwendigerweise auf die Schnittmenge der unterstützten Frameworks beschränkt.

Kapitel 6 beantwortet FF3 mit einem zweigeteilten Befund. Gegenüber *PennyLane* entsteht in der Ausführung kein messbarer Overhead, da der *Executor* eine vorbereitete Instruktionsliste abspielt, während die native Seite den Schaltkreis bei jedem Aufruf neu aufzeichnet. Gegenüber *Qiskit* kostet die Abstraktion dagegen messbar, und zwar um den Faktor 1,97 bei der Erstellung, 1,40 bei der Ausführung und 2,40 bei der Gradientenberechnung. Der relative Overhead der Ausführung nimmt mit wachsender Qubit-Zahl ab, da der Zusatzaufwand aus reiner Python-Buchhaltung besteht und nicht von der Registergröße abhängt. Allein bei der Gradientenberechnung über *Qiskit* wächst der Faktor mit der Gatterzahl, was auf die tiefen Schaltkreiskopien der Parameter-Shift-Regel zurückgeht. Der gemessene Vorsprung des *Executors* bei der Gradientenberechnung über *PennyLane* bleibt wegen abweichender Bibliotheksversionen zwischen beiden Messläufen nur eingeschränkt belastbar.

Der gemessene Mehraufwand fällt gegenüber dem gewonnenen Nutzen gering aus. Gegenüber *PennyLane* arbeitet der *Executor* auf dem Niveau der nativen API, und gegenüber *Qiskit* bewegt sich der Overhead in der Ausführung in einem Bereich, der sich mit wachsender Registergröße weiter angleicht, da der Zusatzaufwand nicht mit der Qubit-Zahl skaliert. Er beschränkt sich damit auf einen konstanten Anteil an Python-Arbeit, dem eine einmalige, über sämtliche Backends hinweg wiederverwendbare Problembeschreibung gegenübersteht. Gerade dort, wo Laufzeit tatsächlich ins Gewicht fällt, also bei großen Registern und rechenintensiven Simulationen, verliert der Overhead zusätzlich an Bedeutung. Die in Kapitel 5 beschriebenen Vorteile, insbesondere die kontrollierte Vergleichbarkeit von Backends, die optionalen Framework-Abhängigkeiten und die Stabilität über Versionswechsel hinweg, wiegen diesen Anteil in der Praxis auf.

Damit sind alle drei Forschungsfragen beantwortet.

7.2. Ausblick

Der Executor im Allgemeinen und die Abstraktionsschicht können noch weiterentwickelt werden, um Anbindungen an weitere Backends zu schaffen. Im Moment werden nur *PennyLane* und *Qiskit* als API-Endpunkte unterstützt, was bereits eine gute Basis bildet. Darüber hinaus ist die Grundlage für eine Implementierung von *Qulacs* und der Pauli-Propagation bereits gegeben. Bei diesen fehlt jedoch noch die Umwandlung der abstrakten Schaltkreis- und Observablen-Objekte in die Framework-spezifischen Repräsentationen. Auch weitere Frameworks wie *Cirq*, *Braket SDK* oder *Q#* wären sinnvolle Ergänzungen. Damit würde der Executor ein größeres Ökosystem unterstützen und mehr Nutzerinnen und Nutzer ansprechen, die diese Backends einsetzen.

Zudem orientiert sich die Abstraktionsschicht derzeit an der NISQ-Ära, also an verrauschten Quantencomputern ohne Fehlerkorrektur [15]. Ihr Entwurf ist daher auf parametrisierte Schaltkreise sowie die Berechnung von Erwartungswerten und Gradienten ausgelegt. Perspektivisch wird jedoch das fehlertolerante Quantencomputing wichtiger, das mit fehlerkorrigierten logischen Qubits arbeitet. Diese Technologie steht allerdings noch am Anfang und ist bisher nicht praktisch nutzbar, sodass eine Anpassung der Abstraktionsschicht erst mit der Verfügbarkeit entsprechender Backends sinnvoll wird.

Literaturverzeichnis

[1] K. Bashiri, *Quantencomputing, Grundlagen, Algorithmen, Anwendung*. Bonn: Rheinwerk Computing, 2025, ISBN: 978-3-8362-9848-3.

[2] A. G. Baydin, B. A. Pearlmutter, A. A. Radul und J. M. Siskind, „Automatic Differentiation in Machine Learning: A Survey," *Journal of Machine Learning Research*, Jg. 18, Nr. 153, S. 1–43, 2018.

[3] V. Bergholm, J. Izaac, M. Schuld, C. Gogolin, M. S. Alam, S. Ahmed, J. M. Arrazola, C. Blank, A. Delgado, S. Jahangiri, K. McKiernan, J. J. Meyer, Z. Niu, A. Száva, N. Killoran u. a., „PennyLane: Automatic Differentiation of Hybrid Quantum-Classical Computations," 2018. DOI: 10.48550/arXiv.1811.04968 arXiv: 1811.04968 [quant-ph].

[4] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe und S. Lloyd, „Quantum machine learning," *Nature*, Jg. 549, Nr. 7671, S. 195–202, 2017. DOI: 10.1038/nature23474

[5] F. Bova, A. Goldfarb und R. G. Melko, „Commercial applications of quantum computing," *EPJ Quantum Technology*, Jg. 8, Nr. 1, S. 2, 2021. DOI: 10.1140/epjqt/s40507-021-00091-1

[6] D. Deutsch, „Quantum theory, the Church–Turing principle and the universal quantum computer," *Proceedings of the Royal Society A*, Jg. 400, Nr. 1818, S. 97–117, 1985. DOI: 10.1098/rspa.1985.0070

[7] M. Fingerhuth, T. Babej und P. Wittek, „Open source software in quantum computing," *PLOS ONE*, Jg. 13, Nr. 12, e0208561, 2018. DOI: 10.1371/journal.pone.0208561

[8] E. Gibney, „Quantum gold rush: the private funding pouring into quantum start-ups," *Nature*, Jg. 574, Nr. 7776, S. 22–24, 2019. DOI: 10.1038/d41586-019-02935-4

[9] M. Homeister, *Quantum Computing verstehen, Grundlagen – Anwendungen – Perspektiven*, 6. Aufl. Wiesbaden: Springer Vieweg, 2022. DOI: 10.1007/978-3-658-36434-2

[10] IBM Quantum. „Introduction to IBM Quantum services,“ besucht am 16. Juli 2026. Adresse: https://quantum.cloud.ibm.com/docs/en/guides/compute-services

[11] IBM Quantum. „Qiskit Runtime (`qiskit-ibm-runtime`),“ besucht am 16. Juli 2026. Adresse: https://github.com/Qiskit/qiskit-ibm-runtime

[12] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson und J. M. Gambetta, *Quantum computing with Qiskit*, 2024. DOI: 10.48550/arXiv.2405.08810 arXiv: 2405.08810 [quant-ph].

[13] K. Mitarai, M. Negoro, M. Kitagawa und K. Fujii, „Quantum circuit learning,“ *Physical Review A*, Jg. 98, Nr. 3, S. 032 309, Sep. 2018. DOI: 10.1103/PhysRevA.98.032309 Adresse: https://link.aps.org/doi/10.1103/PhysRevA.98.032309

[14] M. A. Nielsen und I. L. Chuang, *Quantum Computation and Quantum Information, 10th Anniversary Edition*, 10. Aufl. Cambridge: Cambridge University Press, 2010, ISBN: 9781107002173.

[15] J. Preskill, „Quantum Computing in the NISQ Era and Beyond,“ *Quantum*, Jg. 2, S. 79, 2018. DOI: 10.22331/q-2018-08-06-79

[16] R. Santagati, A. Aspuru-Guzik, R. Babbush, M. Degroote, L. González, E. Kyoseva, N. Moll, M. Oppel, R. M. Parrish, N. C. Rubin, M. Streif, C. S. Tautermann, H. Weiss, N. Wiebe und C. Utschig-Utschig, „Drug design on quantum computers,“ *Nature Physics*, Jg. 20, Nr. 4, S. 549–557, 2024. DOI: 10.1038/s41567-024-02411-5

[17] E. Schrödinger, „Die gegenwärtige Situation in der Quantenmechanik,“ *Die Naturwissenschaften*, Jg. 23, Nr. 48, S. 807–812, 1935. DOI: 10.1007/BF01491891

[18] M. Schuld, V. Bergholm, C. Gogolin, J. Izaac und N. Killoran, „Evaluating Analytic Gradients on Quantum Hardware,“ *Physical Review A*, Jg. 99, Nr. 3, S. 032 331, 2019. DOI: 10.1103/PhysRevA.99.032331

[19] M. Schuld, V. Bergholm, C. Gogolin, J. Izaac und N. Killoran, „Evaluating analytic gradients on quantum hardware,“ *Physical Review A*, Jg. 99, Nr. 3, S. 032 331, März 2019. DOI: 10.1103/PhysRevA.99.032331 Adresse: https://link.aps.org/doi/10.1103/PhysRevA.99.032331

[20] H. Taherdoost, „Quantum Computing,“ in *Innovating and Integrating Emerging Technologies*. Cham: Springer Nature Switzerland, 2025, S. 83–91. DOI: 10.1007/978-3-031-93953-2_7

[21] P.-H. Wang, W.-Y. Wu, C.-Y. Lee, J.-C. Hong und Y. J. Tseng, „Advantages of two quantum programming platforms in quantum computing and quantum chemistry,“ *Journal of Cheminformatics*, Jg. 17, Nr. 1, S. 77, Mai 2025, ISSN: 1758-2946. DOI: 10.1186/s13321-025-01026-z Adresse: https://doi.org/10.1186/s13321-025-01026-z

A. Anhang

A.1. Zugang zum Quellcode

Der im Rahmen dieser Arbeit entwickelte Quellcode ist öffentlich über GitHub verfügbar und gliedert sich in zwei Repositories.

Das erste Repository enthält die Implementierung der Abstraktionsschicht `qc_executor`, welche die einheitliche Ansteuerung der verschiedenen Backends kapselt:

https://github.com/flaqship/qc-executor

Das zweite Repository umfasst das Benchmarking-System, mit dem die vorgestellten Performance-Messungen durchgeführt wurden:

https://github.com/dfaigle/benchmarking-system

A.2. Eidesstattliche Erklärung

Ich erkläre hiermit an Eides statt, dass ich die vorliegende Bachelor-Thesis mit dem Titel

Entwicklung und Benchmarking einer Abstraktionsschicht für Quanten-Software

selbstständig und ohne Benutzung anderer als der angegebenen Hilfsmittel angefertigt habe.

Bei der Erstellung dieser Arbeit wurden KI-basierte Werkzeuge unterstützend eingesetzt, und zwar zur Rechtschreib- und Formulierungshilfe, beim Finden von Quellen sowie zur Unterstützung beim Programmieren. Die inhaltliche Verantwortung für die Arbeit liegt vollständig bei mir.

Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder nicht veröffentlichten Schriften entnommen wurden, sind als solche kenntlich gemacht. Die Arbeit ist in gleicher oder ähnlicher Form noch nicht als Prüfungsarbeit eingereicht worden.

Ich bin mir bewusst, dass eine falsche Erklärung rechtliche Folgen haben wird.

Heilbronn, August 2026

Ort, Datum	Daniel Faigle